

# **RTL Design Sherpa**

## **DDR2 / LPDDR2 Family Controller Micro-Architecture Specification 0.1**

**June 13, 2026**

## Table of Contents

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 1 Ddr2 Lpddr2 Mas Index.....                                              | 13 |
| 2 Document Information.....                                               | 13 |
| 2.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification..... | 13 |
| 2.2 Document Purpose.....                                                 | 13 |
| 2.3 Document Scope.....                                                   | 14 |
| 2.4 Revision History.....                                                 | 14 |
| 3 Architecture and Datapath.....                                          | 15 |
| 3.1 Top-Level FUB Topology.....                                           | 15 |
| 3.2 Read / Write Datapath: AXI → addr-hash → CAM.....                     | 16 |
| 3.2.1 Why Two CAMs, Not One.....                                          | 16 |
| 3.2.2 Why the XOR Happens Before the CAM.....                             | 18 |
| 3.2.3 Why the AXI-side metadata stays in axi4_slave_fub.....              | 18 |
| 3.3 Clocking Topology.....                                                | 18 |
| 3.4 DFI Phase / Gear Topology.....                                        | 19 |
| 4 Top-Level Port List.....                                                | 19 |
| 4.1 Parameter-Dependent Widths (re-stated).....                           | 20 |
| 4.2 AXI4 Slave Port List.....                                             | 20 |
| 4.3 DFI v2.1 Master Port List.....                                        | 20 |
| 4.4 APB CSR Slave Port List.....                                          | 20 |
| 4.5 Clocks, Resets, IRQs, Debug.....                                      | 21 |
| 4.6 SystemVerilog Header Snippet.....                                     | 21 |
| 5 Clocks and Reset.....                                                   | 22 |

|                                                    |    |
|----------------------------------------------------|----|
| 5.1 Clock Domains.....                             | 22 |
| 5.2 Reset Topology.....                            | 22 |
| 5.3 CDC.....                                       | 23 |
| 5.4 Quiet Points.....                              | 23 |
| 5.5 Reset Sequence.....                            | 24 |
| 6 Top-Level Integration (ddr2_lpddr2_ctrl).....    | 24 |
| 6.1 Purpose.....                                   | 24 |
| 6.2 Instantiation Inventory.....                   | 25 |
| 6.3 Generate Blocks.....                           | 26 |
| 6.4 What the Top Does Not Do.....                  | 26 |
| 7 AXI4 Slave (axi4_slave_fub).....                 | 27 |
| 7.1 Purpose.....                                   | 27 |
| 7.2 External Interface (AXI side).....             | 27 |
| 7.3 Internal Buffers.....                          | 27 |
| 7.4 Data Flow.....                                 | 28 |
| 7.4.1 Write Path.....                              | 28 |
| 7.4.2 Read Path.....                               | 28 |
| 7.5 Outstanding Burst Tracking.....                | 29 |
| 7.6 Timing Budget.....                             | 29 |
| 7.7 CSR Hooks.....                                 | 30 |
| 7.8 TODO (week-of-MAS work).....                   | 30 |
| 8 Address XOR / Hash (addr_mapper_fub).....        | 30 |
| 8.1 Purpose.....                                   | 30 |
| 8.2 Mirror to the Python AddressMapping Class..... | 31 |

|                                                |    |
|------------------------------------------------|----|
| 8.3 Interface.....                             | 31 |
| 8.4 Scheme Decode.....                         | 31 |
| 8.4.1 Scheme 1: ROW_MAJOR.....                 | 31 |
| 8.4.2 Scheme 2: BANK_INTERLEAVE.....           | 32 |
| 8.4.3 Scheme 3: XOR_HASH.....                  | 32 |
| 8.5 Runtime Scheme Mux.....                    | 32 |
| 8.6 Timing Budget.....                         | 32 |
| 8.7 Verification.....                          | 33 |
| 8.8 TODO.....                                  | 33 |
| 9 Read Command CAM (rd_cmd_cam_fub).....       | 33 |
| 9.1 Purpose.....                               | 34 |
| 9.2 Parameters.....                            | 34 |
| 9.3 Storage Per Slot.....                      | 34 |
| 9.4 Interfaces.....                            | 35 |
| 9.4.1 Push (from axi4_slave_fub).....          | 35 |
| 9.4.2 Scheduler Query.....                     | 35 |
| 9.4.3 Issue Notification.....                  | 35 |
| 9.4.4 Beat Return (from rd_data_path_fub)..... | 36 |
| 9.5 Lookup Mechanism.....                      | 36 |
| 9.6 Per-ID Beat Counting.....                  | 36 |
| 9.7 Timing Budget.....                         | 36 |
| 9.8 CSR Hooks.....                             | 37 |
| 9.9 TODO.....                                  | 37 |
| 10 Write Command CAM (wr_cmd_cam_fub).....     | 37 |

|                                                           |    |
|-----------------------------------------------------------|----|
| 10.1 Purpose.....                                         | 37 |
| 10.2 Parameters.....                                      | 38 |
| 10.3 Storage Per Slot.....                                | 38 |
| 10.4 Interfaces.....                                      | 39 |
| 10.4.1 Push (from axi4_slave_fub).....                    | 39 |
| 10.4.2 Scheduler Query.....                               | 39 |
| 10.4.3 Issue Notification.....                            | 39 |
| 10.4.4 Beat-Pull (to wr_data_path_fub).....               | 39 |
| 10.4.5 Completion (from DFI write window expiration)..... | 39 |
| 10.5 Difference From Read CAM.....                        | 39 |
| 10.6 Timing Budget.....                                   | 40 |
| 10.7 CSR Hooks.....                                       | 40 |
| 10.8 TODO.....                                            | 40 |
| 11 Transaction Queue (txn_queue_fub).....                 | 41 |
| 11.1 Purpose.....                                         | 41 |
| 11.2 Relationship to the CAMs.....                        | 41 |
| 11.3 Synthesis Parameters.....                            | 42 |
| 11.4 Entry Format.....                                    | 42 |
| 11.5 Entry Lifecycle.....                                 | 44 |
| 11.6 Ports.....                                           | 45 |
| 11.6.1 Insert (from CAMs).....                            | 45 |
| 11.6.2 Parallel Read (to scheduler — every cycle).....    | 46 |
| 11.6.3 Bank-State Watchpoint (from bank machines).....    | 46 |
| 11.6.4 Scheduler Pick (from scheduler).....               | 46 |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 11.6.5 CAM Completion (from CAMs).....                                            | 47 |
| 11.6.6 Backpressure Output.....                                                   | 47 |
| 11.7 Insert Path.....                                                             | 47 |
| 11.8 Row-Hit Cache Coherency.....                                                 | 48 |
| 11.9 Age Counter Saturation.....                                                  | 49 |
| 11.10 Storage Choice.....                                                         | 49 |
| 11.11 CSR Hooks.....                                                              | 50 |
| 11.12 Verification Notes (cocotb test plan).....                                  | 50 |
| 11.13 Open Questions / Future Work.....                                           | 51 |
| 12 FR-FCFS Scheduler (scheduler_fub).....                                         | 52 |
| 12.1 Purpose.....                                                                 | 52 |
| 12.2 Synthesis Parameters.....                                                    | 52 |
| 12.3 Interface.....                                                               | 53 |
| 12.3.1 Inputs.....                                                                | 53 |
| 12.3.2 Outputs.....                                                               | 54 |
| 12.4 Decision Pipeline.....                                                       | 55 |
| 12.4.1 Stage 1 — Per-Entry Readiness (combinational, N parallel comparators)..... | 56 |
| 12.4.2 Stage 2 — Priority Masking (combinational).....                            | 57 |
| 12.4.3 Stage 3 — Selection (combinational priority encoders).....                 | 58 |
| 12.4.4 Stage 4 — Command Formation (registered).....                              | 58 |
| 12.4.5 Refresh Window Behavior.....                                               | 59 |
| 12.5 Cache Coherency: row_hit_cached.....                                         | 59 |
| 12.6 Pipeline Staging.....                                                        | 60 |

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 12.7 Strict In-Order Mode (collapse).....                                     | 60 |
| 12.8 Critical-Path Analysis (rough budget at 500 MHz target, 2 ns cycle)..... | 61 |
| 12.9 CSR Hooks.....                                                           | 61 |
| 12.10 Verification Notes (cocotb test plan).....                              | 62 |
| 12.11 Open Questions / Future Work.....                                       | 63 |
| 13 HAPPY Page Predictor (page_predictor_fub).....                             | 63 |
| 13.1 TODO (Week-of-MAS work).....                                             | 64 |
| 14 Bank Machine (bank_machine_fub).....                                       | 64 |
| 14.1 Purpose.....                                                             | 64 |
| 14.2 Synthesis Parameters.....                                                | 65 |
| 14.3 Instantiation Pattern.....                                               | 65 |
| 14.4 Block Internals.....                                                     | 67 |
| 14.5 FSM States and Transition Triggers.....                                  | 68 |
| 14.6 Counter Pool.....                                                        | 69 |
| 14.7 Outputs to Scheduler (accepts_*)......                                   | 70 |
| 14.8 Refresh Handshake Protocol.....                                          | 70 |
| 14.9 Open-Row Change Broadcast.....                                           | 71 |
| 14.10 Per-Instance vs. Aggregated Storage.....                                | 72 |
| 14.11 Per-Instance Area Estimate.....                                         | 73 |
| 14.12 CSR Hooks.....                                                          | 73 |
| 14.13 Verification Notes (cocotb test plan).....                              | 74 |
| 14.14 Open Questions / Future Work.....                                       | 75 |
| 15 Cross-Bank Timers (xbank_timers_fub).....                                  | 75 |
| 15.1 TODO (Week-of-MAS work).....                                             | 76 |

|                                            |    |
|--------------------------------------------|----|
| 16 Refresh Manager (refresh_mgr_fub).....  | 76 |
| 16.1 TODO (Week-of-MAS work).....          | 76 |
| 17 Init Engine (init_engine_fub).....      | 77 |
| 17.1 TODO (Week-of-MAS work).....          | 77 |
| 18 Power-State FSM (power_state_fub).....  | 77 |
| 18.1 TODO (Week-of-MAS work).....          | 77 |
| 19 Command Encoder (cmd_encoder_fub).....  | 78 |
| 19.1 TODO (Week-of-MAS work).....          | 78 |
| 20 DFI Gear (gear_dfi_fub).....            | 78 |
| 20.1 TODO (Week-of-MAS work).....          | 79 |
| 21 ODT Control (odt_ctrl_fub).....         | 79 |
| 21.1 TODO (Week-of-MAS work).....          | 79 |
| 22 Write Data Path (wr_data_path_fub)..... | 80 |
| 22.1 TODO (Week-of-MAS work).....          | 80 |
| 23 Read Data Path (rd_data_path_fub).....  | 80 |
| 23.1 TODO (Week-of-MAS work).....          | 80 |
| 24 AXI4 Slave Protocol.....                | 81 |
| 24.1 Scope of This Section.....            | 81 |
| 24.2 Supported Features (v1).....          | 81 |
| 24.3 Optional Features (build-time).....   | 81 |
| 24.4 Unsupported Features.....             | 81 |
| 24.5 TODO.....                             | 81 |
| 25 DFI v2.1 Master Protocol.....           | 82 |
| 25.1 Scope.....                            | 82 |



|                                                          |    |
|----------------------------------------------------------|----|
| 25.2 Sub-Interfaces Supported.....                       | 82 |
| 25.3 Sub-Interfaces Not Supported (v1).....              | 82 |
| 25.4 Phase / Gear Topology.....                          | 82 |
| 25.5 TODO.....                                           | 83 |
| 26 APB CSR Slave Protocol.....                           | 83 |
| 26.1 Protocol Compliance.....                            | 83 |
| 26.2 Behavior.....                                       | 83 |
| 26.3 CDC.....                                            | 83 |
| 26.4 TODO.....                                           | 83 |
| 27 Register Map.....                                     | 84 |
| 27.1 Source of Truth.....                                | 84 |
| 27.2 Staging vs. Live State.....                         | 84 |
| 27.3 TODO.....                                           | 85 |
| 28 Runtime Overrides and Quiet Points.....               | 85 |
| 28.1 Override Categories.....                            | 85 |
| 28.2 Quiet-Point Sequence.....                           | 86 |
| 28.3 TODO.....                                           | 86 |
| 29 Family-Wide Config-Bit Applicability.....             | 86 |
| 29.1 Per-Bit Implementation Notes for DDR2 / LPDDR2..... | 86 |
| 29.2 Soft-N/A Behavior Examples.....                     | 87 |
| 29.3 TODO.....                                           | 87 |
| 30 Initialization Sequence.....                          | 87 |
| 30.1 TODO.....                                           | 87 |
| 31 Refresh and Power-State Programming.....              | 88 |

|                                            |    |
|--------------------------------------------|----|
| 31.1 TODO.....                             | 88 |
| 32 Multi-Rank Programming.....             | 88 |
| 32.1 TODO.....                             | 88 |
| 33 Error Handling.....                     | 89 |
| 33.1 TODO.....                             | 89 |
| 34 Build-Time Configuration Reference..... | 89 |
| 34.1 TODO.....                             | 90 |
| 35 Runtime Configuration Reference.....    | 90 |
| 35.1 TODO.....                             | 90 |

## List of Figures

*No figures in this document.*

## List of Tables

*No tables in this document.*

# 1 Ddr2 Lpddr2 Mas Index

Generated: 2026-06-16

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 2 Document Information

### 2.1 DDR2 / LPDDR2 Family Controller Micro-Architecture Specification

---

**Document Number:** DDR2-LPDDR2-MAS-001 **Version:** 0.1 **Status:** Draft - Skeleton **Classification:** Open Source - Apache 2.0 License

---

### 2.2 Document Purpose

---

This Micro-Architecture Specification (MAS) is the implementation-level companion to the DDR2 / LPDDR2 Hardware Architecture Specification (HAS). Where the HAS answers “what is the architecture and why,” this MAS answers “what is in the RTL and how does it work.”

The MAS is the document RTL designers, verification engineers, and integrators consult when writing or reviewing SystemVerilog code, building testbenches, and stitching the controller into an SoC.

#### Target Audience:

- RTL designers implementing the FUBs described in this document
- Verification engineers writing per-FUB cocotb tests
- Integrators stitching the controller into an SoC
- Software engineers writing low-level memory configuration code (driver authors, bring-up engineers)

#### Companion Documents:

- DDR2/LPDDR2 Family Controller HAS ([./ddr2\\_lpddr2\\_has/](#)) — high-level architecture and design rationale

- DDR2/LPDDR2 DFI BFM documentation (in RTLDesignSherpa-DV) — verification-side reference
- 

## 2.3 Document Scope

---

The MAS is the **micro-architecture** view: per-FUB inputs/outputs, internal state, FSM diagrams, datapath timing, register-level pipeline stages, and per-block timing budgets. It assumes the reader has read the HAS for context.

This document covers:

- Top-level integration wiring (ddr2\_lpddr2\_ctrl)
- Each FUB's interface signal list, internal storage, datapath flow, FSM, and timing budget
- AXI4 and DFI v2.1 wire-level protocol details specific to this controller
- APB programming interface and the full CSR register map
- Programming sequences (init, refresh, power-down, multi-rank, error handling)
- Build-time and runtime configuration reference

This document does **not** cover:

- Higher-level architectural rationale (see HAS)
  - Verification plans or coverage models (see dv/README.md)
  - Floorplan or layout guidance (project-specific)
  - Bit-level CSR YAML — that lives in regs/ and is the source of truth for CSR generation
- 

## 2.4 Revision History

---

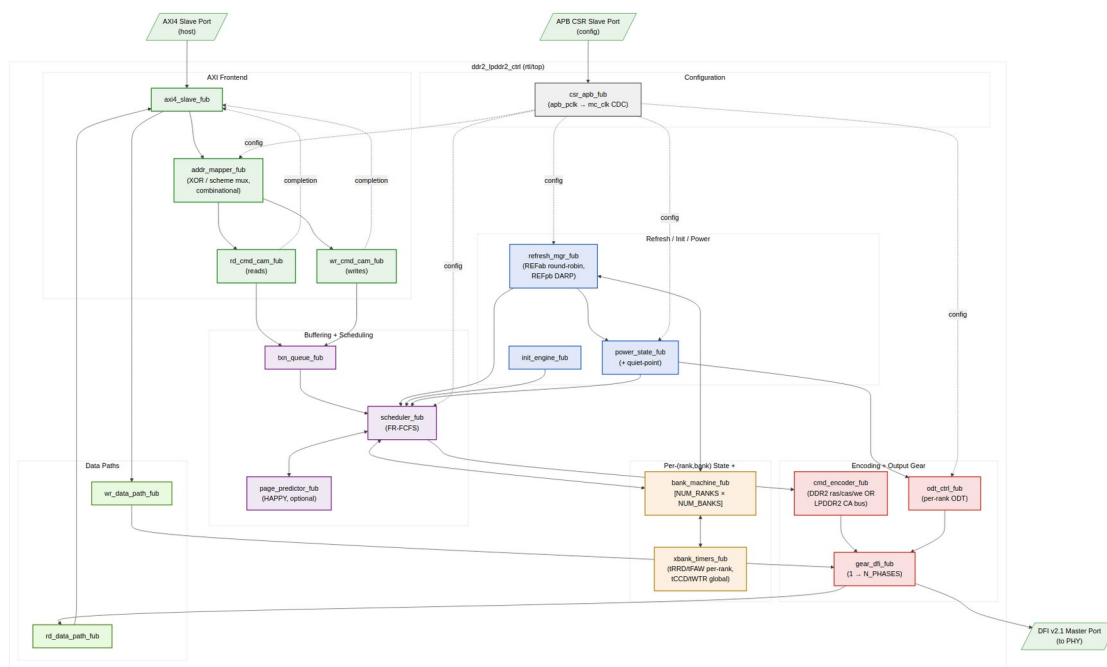
| Version | Date       | Author            | Notes                                                            |
|---------|------------|-------------------|------------------------------------------------------------------|
| 0.1     | 2026-06-14 | RTL Design Sherpa | Initial skeleton — chapter outline, FUB list, port list scaffold |

## 3 Architecture and Datapath

This chapter is the implementation-level architectural orientation: the top-level FUB stitching, the read/write datapath flow, and the clocking topology. Per-FUB detail is in §2.

### 3.1 Top-Level FUB Topology

The controller's top is `ddr2_lpddr2_ctrl.sv` — a pure structural integration of leaf FUBs. The 18 leaf FUBs are partitioned into 7 functional groups (see HAS §2.5 for the SWAG-level group description; this section is the wire-level topology view).



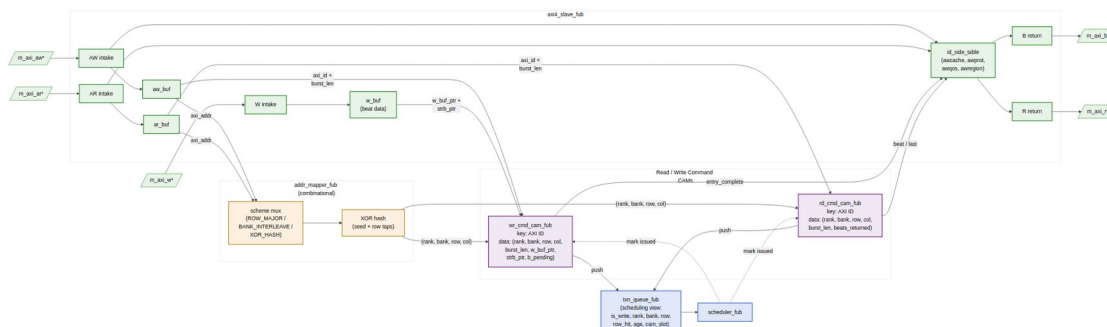
#### Top-Level FUB Topology

Source: [01\\_top\\_fub\\_topology.mmd](#)

The salient property: there is exactly **one** stage where AXI-layer concepts (burst, ID, write strobe) cross into DRAM-layer concepts (rank, bank, row, column). That stage is `addr_mapper` → `{rd,wr}_cmd_cam`. Upstream of `addr_mapper` everything is AXI; downstream of the CAMs everything is DRAM.

## 3.2 Read / Write Datapath: AXI → addr-hash → CAM

The user-facing entry point is `axi4_slave_fub`. From the slave, the read and write paths share `addr_mapper` (one cycle) and then split into two separate CAMs:



*AXI → addr\_mapper → CAMs Datapath*

Source: [02\\_axi\\_to\\_cam\\_datapath.mmd](#)

### 3.2.1 Why Two CAMs, Not One

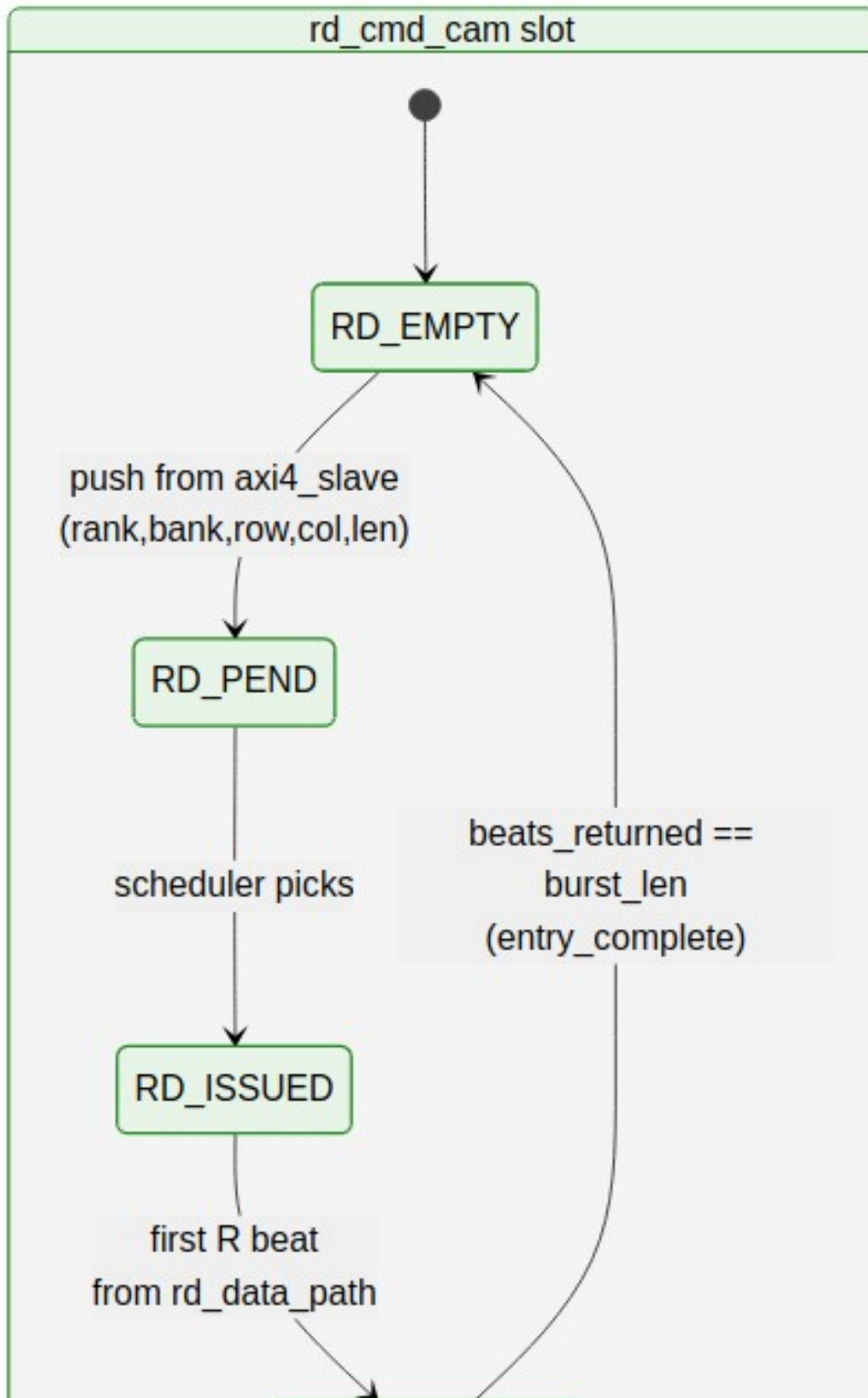
The CAM split happens here, not at the scheduler, because read and write CAMs hold **different metadata** and have **different lifetimes**:

| CAM        | Key    | Data                                                                   | Cleared on               |
|------------|--------|------------------------------------------------------------------------|--------------------------|
| wr_cmd_cam | AXI ID | (rank, bank, row, col, burst_len, <b>w_buf_ptr, strb_ptr</b> )         | B-channel response push  |
| rd_cmd_cam | AXI ID | (rank, bank, row, col, burst_len, <b>rid_counter, expected_beats</b> ) | last R beat of the burst |

A single unified CAM would either carry both metadata sets (wasteful) or force an awkward “is\_write” predicate on every lookup. Keeping the CAMs separate also means the read and write paths can be sized independently — the read path typically deeper because read latency is longer; the write path can be shallower because write completion is fire-and-forget.

The two CAMs have visibly different lifecycles — note in particular that the write side stays “alive” through the post-issue write window (waiting on `b_complete` from `xbank_timers`) while the read side just counts beats:





## CAM Slot Lifecycles (read vs. write)

Source: [03\\_cam\\_lifecycles.mmd](#)

### 3.2.2 Why the XOR Happens Before the CAM

Address XOR / hash (`addr_mapper`) **must** happen before the CAM push because the CAM stores the post-decode (rank, bank, row, col) tuple — not the raw AXI address. Two reasons:

1. **Stable CAM key data.** The CAM is queried by the scheduler in the (rank, bank) dimension — for example, “do any pending writes target rank 0 bank 3, row R?” Storing the post-decode tuple makes this a direct comparison; storing the raw AXI address would force the scheduler to re-XOR on every match.
2. **Address-mapping scheme is per-controller, not per-burst.** The `ADDR_MAP_TUNING.scheme_or` field is global to the controller. If we stored raw addresses, a runtime scheme change (which can happen at any quiet point) would force a CAM walk to re-decode every entry. Storing the decoded tuple means a scheme change only affects **new** AXI bursts — old in-flight bursts continue with their decoded address. This matches the quiet-point semantics in HAS §6.3.

The `addr_mapper` is **combinational** — no pipeline stage between AXI intake and CAM push — so the timing budget for the XOR network is one cycle minus the AXI register-slice slack. See `ch02_blocks/03_addr_mapper.md` for the timing breakdown.

### 3.2.3 Why the AXI-side metadata stays in `axi4_slave_fub`

Some AXI fields (`awcache`, `awprot`, `awqos`, `awregion`, `bid/rid`) never need to cross into DRAM-layer state. These are held in small **side tables** inside `axi4_slave_fub`, indexed by the same AXI ID that keys the CAMs. The CAM holds the *DRAM* state; the side table holds the *AXI* state. On B/R completion, the CAM produces the completion signal and the side table produces the response metadata; the two are joined at the AXI return port.

This split keeps the CAM narrow (16 ID slots × ~36 bits in the default config) and pushes the AXI-specific clutter into the FUB that already owns the protocol.

## 3.3 Clocking Topology

Two external clocks; one CDC.

| Clock                 | Frequency Range | Drives                                                                                                                                                                                  |
|-----------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>mc_clk</code>   | 100–500 MHz     | All DRAM-layer FUBs: scheduler, bank machines, refresh, init, power, <code>cmd_encoder</code> , <code>gear_dfi</code> , <code>odt_ctrl</code> , data paths, and <code>axi4_slave</code> |
| <code>apb_pclk</code> | 25–100 MHz      | CSR slave only                                                                                                                                                                          |

| Clock | Frequency Range | Drives |
|-------|-----------------|--------|
|-------|-----------------|--------|

`csr_apb_fub` owns the single CDC between `apb_pclk` and `mc_clk`. CSR overrides are handed off across this CDC into a quiet-point staging register; the actual override-application happens at the next quiet point on `mc_clk`. Quiet-point detection is in `power_state_fub` (no commands in flight, no refresh in progress).

There is **no CDC** in the AXI4 datapath — `axi4_slave_fub` runs in the `mc_clk` domain. If the SoC's AXI master is on a different clock, an external clock converter is required upstream of the controller.

### 3.4 DFI Phase / Gear Topology

The DFI v2.1 master interface is `N_PHASES`-deep. The MC clock drives one frame of `N_PHASES` DFI cycles per MC clock cycle. The `gear_dfi_fub` rewinds scheduler-issued commands and AXI data beats into the right per-phase slot per `WRPHASE` / `RDPHASE`. The scheduler does not see the phase dimension — its issue rate is one command per MC clock; the gear absorbs all of the phase-level scheduling.

The block diagrams and FSM diagrams referenced inline above are in `assets/mermaid/` (TBD; placeholders until the per-block detail in §2 is complete).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

## 4 Top-Level Port List

This chapter is the **wire-level** port list of the `ddr2_lpddr2_ctrl` top module — the exact signal names, directions, widths, and parameter dependencies that the integration script and SoC top-level use.

The implementation-level signal list is the same as the HAS overview view (HAS §2.4); this chapter restates it in the SystemVerilog-derivable form, with notes on bit-ordering, default tie-offs, and synthesis attributes where they matter.

The canonical machine-generated source is `regs/ddr2_lpddr2_ctrl_ports.svh` — what follows is the human-readable mirror.

## 4.1 Parameter-Dependent Widths (re-stated)

---

- $AW = AXI\_ADDR\_WIDTH$
- $DW = AXI\_DATA\_WIDTH$
- $SW = DW/8$
- $IDW = AXI\_ID\_WIDTH$
- $DFI\_DW = DFI\_DATA\_WIDTH$
- $DFI\_AW = DFI\_ADDR\_WIDTH$
- $NP = N\_PHASES$
- $NR = NUM\_RANKS$
- $NB = NUM\_BANKS$
- $BA = \lceil \log_2(NUM\_BANKS) \rceil$

## 4.2 AXI4 Slave Port List

---

See HAS §2.4 §1 for the table. The MAS adds nothing beyond what the HAS already published; the SystemVerilog declaration uses the same field names directly. AXI side-band signals (*awqos*, *awregion*, *arqos*, *arregion*) are observed but do not currently drive scheduler priority in v1; the wiring exists for the v2 QoS feature.

## 4.3 DFI v2.1 Master Port List

---

See HAS §2.4 §2. Additional MAS-level notes:

- The *dfi\_cs\_n*[NR] packed dimension is **outer-most** in SV declaration: output [NP-1:0][NR-1:0] *dfi\_cs\_n*;
- Per-rank *dfi\_cke*[NR] is **outer-most**: output [NR-1:0] *dfi\_cke*;
- The DDR2 *ras/cas/we* strobes are present even in LPDDR2 builds (driven all-1s, tied off by synthesis); the elaboration-time MEMTYPE parameter does *not* remove them from the port list, so the same SV header is usable for both flavors.

## 4.4 APB CSR Slave Port List

---

See HAS §2.4 §3. The APB clock and reset (*apb\_pclk*, *apb\_presetn*) are in the misc port group below; only the protocol signals are in this APB block.

## 4.5 Clocks, Resets, IRQs, Debug

See HAS §2.4 §4.

Additional MAS notes:

- `mc_rst_n` is asynchronous-assert, synchronous-deassert inside the controller (the deassert synchronizer lives in `power_state_fub`'s reset domain).
- `apb_presetn` is independently synchronized in `csr_apb_fub`.
- `irq_*` outputs are asserted in the `mc_clk` domain; the SoC's interrupt controller is responsible for re-synchronizing if it runs on a different clock.

## 4.6 SystemVerilog Header Snippet

The first 30 lines of the generated port header — the rest is per the tables above:

```
module ddr2_lpddr2_ctrl #(
    parameter string MEMTYPE           = "DDR2",
    parameter int   N_PHASES           = 4,
    parameter int   NUM_RANKS          = 1,
    parameter int   NUM_BANKS          = 8,
    parameter int   AXI_DATA_WIDTH     = 64,
    parameter int   AXI_ID_WIDTH       = 4,
    parameter int   AXI_ADDR_WIDTH     = 32,
    parameter int   ROW_WIDTH          = 14,
    parameter int   COL_WIDTH          = 10,
    parameter int   DFI_DATA_WIDTH     = 32,
    parameter int   DFI_ADDR_WIDTH     = 14,
    parameter int   WRPHASE            = 0,
    parameter int   RDPHASE            = 0,
    parameter int   TXN_QUEUE_DEPTH    = 16,
    parameter string SCHEDULER_MODE     = "000",
    parameter int   LOOKAHEAD_DEPTH_MAX = 4,
    parameter string PAGE_POLICY        = "HAPPY_HYBRID",
    parameter int   PAGE_PREDICTOR_TABLE_BITS = 12,
    parameter int   AGE_MAX             = 256,
    parameter int   REFRESH_DEFER_MAX  = 8,
    parameter string REFPB_POLICY       = "DARP",
    parameter string ODT_RULE_MULTIRANK = "JEDEC_DDR2",
    parameter int   SIM_INIT_SCALE     = 1
) (
    // Clocks and resets
    input logic mc_clk,
    input logic mc_rst_n,
    input logic apb_pclk,
    input logic apb_presetn,
```

```
// AXI4 slave, DFI master, APB slave, IRQs, debug – see chapter tables.
```

```
// ...  
);
```

The full module declaration is in `regs/ddr2_lpddr2_ctrl_ports.svh` (machine-generated).

## 5 Clocks and Reset

### 5.1 Clock Domains

| Clock    | Frequency   | Polarity | Domain Members                              |
|----------|-------------|----------|---------------------------------------------|
| mc_clk   | 100–500 MHz | Posedge  | All DRAM-layer FUBs, axi4_slave, data paths |
| apb_pclk | 25–100 MHz  | Posedge  | csr_apb_fub only                            |

The DFI v2.1 interface is driven on mc\_clk (the controller side of DFI is always controller-clock; the DFI PHY runs at the DRAM clock and the gear mapping is the gear FUB's responsibility).

### 5.2 Reset Topology

| Reset       | Polarity   | Type                        | Domain   |
|-------------|------------|-----------------------------|----------|
| mc_rst_n    | Active-low | Async assert, sync deassert | mc_clk   |
| apb_presetn | Active-low | Async assert, sync deassert | apb_pclk |

Each reset has its own 2-flop synchronizer for the deassert edge, instantiated in the FUB that owns the domain:

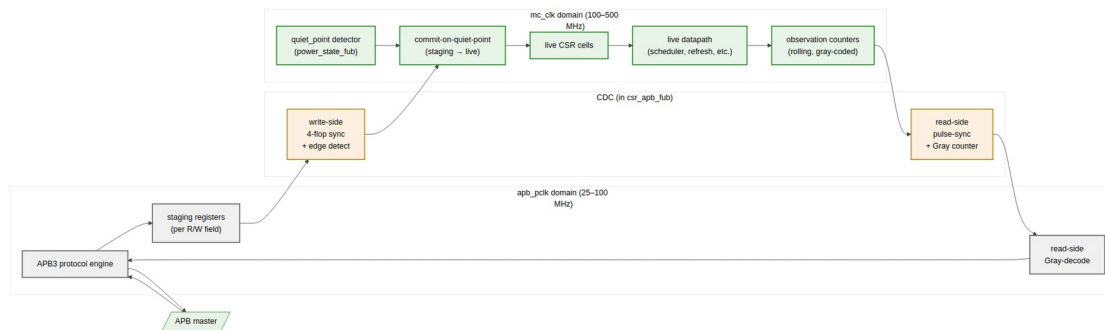
- mc\_rst\_n sync — in power\_state\_fub
- apb\_presetn sync — in csr\_apb\_fub

Both synchronizers are clear-on-async-assert, hold-during-async-low. No reset-glitch filter is in the controller — the SoC's PMU is expected to drive a clean reset.

## 5.3 CDC

Exactly one CDC in the controller: APB → MC for CSR overrides and MC → APB for CSR readback. Both are handled in `csr_apb_fub`. The two directions use different mechanisms:

- **APB → MC** (write side): 4-flop synchronizer per register field, with a per-field `apb_write_strobe` edge-detect on the MC side to latch the new value into the staging register. Staging held until next quiet point.
- **MC → APB** (read side): per-counter pulse synchronizers feed Gray-coded saturating counters; APB reads sample the Gray code and decode locally. The MC-side counters never wrap synchronously between reads.



### Clock Domains and CDC Topology

Source: [04\\_clocks\\_cdc.mmd](#)

## 5.4 Quiet Points

A **configuration quiet point** is the only safe moment to propagate CSR overrides from staging registers into the live datapath. Quiet points are defined by:

- `txn_queue_fub` is empty *or* all pending entries are in PENDING state (no ISSUED/COMPLETING)
- `refresh_mgr_fub` is in IDLE (not WAIT\_BANK\_GNTS, DO\_REFRESH, DO\_ZQCS)
- `power_state_fub` is in ACTIVE (not transitioning)
- No DFI command is in flight (the encoder's per-phase pipeline has drained)

When all four hold, `power_state_fub` asserts `quiet_point`. Override-staging in `csr_apb_fub` watches this signal and commits all queued overrides in one MC-clock edge.

Software triggers quiet-point drain via `CTRL.config_apply`. The CSR slave returns `STATUS.config_settled` when commit is complete.

## 5.5 Reset Sequence

---

On power-on:

1. `mc_rst_n` and `apb_presetn` are asserted (both low). PHY drives `dfi_init_complete = 0`.
2. SoC deasserts `apb_presetn`. `csr_apb_fub` comes out of reset; CSRs are R/W-able. Software loads timing CSRs (MR0..MR3, timings, address-map scheme, refresh tuning).
3. SoC deasserts `mc_rst_n`. All DRAM-layer FUBs come out of reset. Controller idles in **POST\_RESET** state.
4. Software writes `CTRL.init_start = 1`. `init_engine_fub` walks the per-memtype step table, driving `dfi_reset_n`, MR loads, ZQCAL, etc.
5. On completion, `init_engine_fub` asserts `irq_init_done` (one cycle) and `STATUS.init_done = 1`. Controller transitions to **ACTIVE**.
6. AXI traffic from the host is honored.

The minimum gap between `apb_presetn` and `mc_rst_n` deassertion is 16 `mc_clk` cycles (the synchronizer chain depth). Software does not need to wait — the controller stalls at **POST\_RESET** until both resets are released.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 6 Top-Level Integration (`ddr2_lpddr2_ctrl`)

**Module:** `ddr2_lpddr2_ctrl.sv` **Location:** `rtl/top/` **Category:** TOP (Structural only — no behavioral logic) **Status:** Skeleton

---

### 6.1 Purpose

---

`ddr2_lpddr2_ctrl` is the controller's top-level integration. It is **pure structural wiring** — every behavioral statement belongs in a FUB. The role of the top is:



1. Declare the external port list (see §1.2)
2. Instantiate every FUB
3. Wire FUB ↔ FUB interfaces
4. Fan-out per-rank / per-bank / per-phase signals where the FUB count is parametric

## 6.2 Instantiation Inventory

| Instance Name        | FUB                | Count                 | Notes                                       |
|----------------------|--------------------|-----------------------|---------------------------------------------|
| u_axi4_slave         | axi4_slave_fub     | 1                     | Single AXI4 slave port                      |
| u_addr_mapper        | addr_mapper_fub    | 1                     | Combinational; no clock                     |
| u_rd_cmd_cam         | rd_cmd_cam_fub     | 1                     | Read-side CAM                               |
| u_wr_cmd_cam         | wr_cmd_cam_fub     | 1                     | Write-side CAM                              |
| u_txn_queue          | txn_queue_fub      | 1                     | Unified pending queue                       |
| u_scheduler          | scheduler_fub      | 1                     |                                             |
| u_page_predictor     | page_predictor_fub | 0 or 1                | Conditional on PAGE_POLICY == HAPPY_HYBRID  |
| u_bank_machine[r][b] | bank_machine_fub   | NUM_RANKS × NUM_BANKS | generate block, two-dimensional             |
| u_xbank_timers       | xbank_timers_fub   | 1                     | Internally per-rank tRRD/tFAW + global tCCD |
| u_refresh_mgr        | refresh_mgr_fub    | 1                     |                                             |
| u_init_engine        | init_engine_fub    | 1                     |                                             |
| u_power_state        | power_state_fub    | 1                     |                                             |
| u_cmd_encoder        | cmd_encoder_fub    | 1                     | Parameterized by MEMTYPE                    |
| u_gear_dfi           | gear_dfi_fub       | 1                     |                                             |
| u_odt_ctrl           | odt_ctrl_fub       | 1                     |                                             |
| u_wr_data_path       | wr_data_path_fub   | 1                     |                                             |
| u_rd_data_path       | rd_data_path_fub   | 1                     |                                             |

| Instance Name | FUB         | Count | Notes                       |
|---------------|-------------|-------|-----------------------------|
| u_csr_apb     | csr_apb_fub | 1     | Only FUB in apb_pclk domain |

## 6.3 Generate Blocks

The only parametric-fanout instances are the bank machines, fan-out arrays of per-rank CSRs, and per-rank ODT/CKE outputs:

```
generate
  for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
    for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
      bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
        .mc_clk          (mc_clk),
        .mc_rst_n        (mc_rst_n),
        .scheduler_if     (sched_to_bank[r][b]),
        .refresh_if       (refresh_to_bank[r][b]),
        .xbank_if         (xbank_to_bank[r][b]),
        .state_o          (bank_state[r][b]),
        .open_row_o       (bank_open_row[r][b]),
        .last_ref_age_o   (bank_last_ref_age[r][b])
      );
    end
  end
endgenerate
```

The for (genvar r) outer loop is the per-rank dimension; the inner is the per-bank dimension. The aggregation of bank\_state[r][b] into the scheduler input is a structural concatenation only — no behavioral aggregation in the top.

## 6.4 What the Top Does Not Do

- No combinational logic beyond signal renaming or pure concatenation
- No CSR field expansion (the CSR slave handles its own field encoding)
- No clock gating (clock-gate insertion is a synthesis-script concern, not RTL)
- No reset synchronization (handled per-domain in power\_state\_fub and csr\_apb\_fub)
- No assign statements except for tying-off optional DFI signals (DDR2 vs LPDDR2 strobe ties)

This intentional poverty of the top file makes it the obvious place for the structural-only synthesis sanity check: “every line in this file is either an instantiation or a port-mapping; if a line is anything else, it is a bug.”

## 7 AXI4 Slave (axi4\_slave\_fub)

**Module:** axi4\_slave\_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2\_lpddr2\_ctrl  
**Status:** Skeleton

### 7.1 Purpose

axi4\_slave\_fub is the AXI4 host-facing protocol engine. It accepts AW/W/AR transactions from the SoC, hands off the address to addr\_mapper\_fub for decode, and pushes the resulting transaction into the appropriate CAM (wr\_cmd\_cam\_fub or rd\_cmd\_cam\_fub). On B and R returns, it pulls completion signals from the CAMs and merges them with AXI-side metadata to produce protocol-compliant responses.

The FUB owns the AXI ID side table — the per-ID set of fields (awcache, awprot, awqos, awregion, bid echo, rid echo) that never need to cross into the DRAM-layer state.

### 7.2 External Interface (AXI side)

Per HAS §2.4 §1 — full AW/W/B/AR/R channel signal list. No additions in MAS.

### 7.3 Internal Buffers

| Buffer          | Depth                               | Width                                                                            | Purpose                   |
|-----------------|-------------------------------------|----------------------------------------------------------------------------------|---------------------------|
| aw_buf          | TXN_QUEUE_DEPTH/2<br>(default 8)    | AW + IDW + 8(len) + 3(size) + 2(burst) + 4(cache) + 3(prot) + 4(qos) + 4(region) | Pending AW until CAM push |
| w_buf           | TXN_QUEUE_DEPTH × max(burst_length) | DW + SW + 1(last) per beat                                                       | Write data staging        |
| ar_buf          | TXN_QUEUE_DEPTH/2<br>(default 8)    | same AW fields                                                                   | Pending AR until CAM push |
| b_response_fifo | small (4)                           | IDW + 2(resp)                                                                    | Pending B responses       |

| Buffer                      | Depth     | Width                                      | Purpose                                      |
|-----------------------------|-----------|--------------------------------------------|----------------------------------------------|
| r_resp<br>onse_<br>fif<br>o | small (4) | IDW + DW + 2(resp) +<br>1(last) per beat   | Pending R responses                          |
| id_sid<br>e_tabl<br>e       | 2^IDW     | 4(cache) + 3(prot) + 4(qos)<br>+ 4(region) | Per-ID AXI metadata,<br>indexed by awid/arid |

The buffers above are FUB-internal; they do **not** appear in the architecture-level transaction queue (txn\_queue\_fub). The transaction queue holds the DRAM-layer view of pending work; these buffers hold the AXI-layer view of in-flight bursts.

## 7.4 Data Flow

### 7.4.1 Write Path

1. AW intake: capture aw\* fields; push to aw\_buf. Compute outstanding-count and assert awready if aw\_buf not full.
2. W intake: accept w\* beats into w\_buf. Each W beat carries wstrb and wlast. The W path does not wait for AW — W can lead AW or trail it within AXI ordering rules.
3. CAM push: when both aw\_buf head and the matching w\_buf head are present (or when a write-only / address-only burst's prerequisites are satisfied), push to wr\_cmd\_cam\_fub with:
  - key = awid
  - data = (rank, bank, row, col, burst\_len, w\_buf\_ptr, strb\_ptr)
  - where (rank, bank, row, col) come from addr\_mapper\_fub (combinational, same cycle)
4. Scheduler issue: when scheduler picks this entry, the FUB hands w\_data\_path\_fub the w\_buf\_ptr and strb\_ptr for it to walk the data beats.
5. B response: when wr\_cmd\_cam\_fub reports the entry complete (last W beat issued + tDFI write to DFI), the FUB pops the AXI ID, joins with id\_side\_table[id].axi\_metadata, and pushes b\_response\_fifo.
6. B emit: standard B-channel handshake; pop b\_response\_fifo.

### 7.4.2 Read Path

1. AR intake: capture ar\* fields; push to ar\_buf. Assert arready if not full.
2. CAM push: pop ar\_buf head; push to rd\_cmd\_cam\_fub with:
  - key = arid

- data = (rank, bank, row, col, burst\_len, rid\_counter, expected\_beats)
  - where (rank, bank, row, col) come from addr\_mapper\_fub
  - rid\_counter is the AXI ID echo
3. Scheduler issue: when scheduler picks this entry, it commands the issue of RD / RDA. rd\_data\_path\_fub watches for the corresponding DFI rddata beats.
  4. R response: as each R beat completes, rd\_data\_path\_fub strobes rd\_cmd\_cam\_fub to decrement expected\_beats; the CAM emits r\_beat\_to\_axi with rid, beat data, rlast when the last beat arrives.
  5. R emit: standard R-channel handshake; pop r\_response\_fifo.

## 7.5 Outstanding Burst Tracking

The FUB enforces:

| Limit                   | Constraint                                     |
|-------------------------|------------------------------------------------|
| Total in-flight reads   | $\leq$ rd_cmd_cam_fub depth (default 16)       |
| Total in-flight writes  | $\leq$ wr_cmd_cam_fub depth (default 16)       |
| Per-ID in-flight reads  | $\leq$ MAX_PER_ID_READS parameter (default 4)  |
| Per-ID in-flight writes | $\leq$ MAX_PER_ID_WRITES parameter (default 4) |

If AXI\_000\_ACROSS\_IDS == false, the per-ID counts reduce to “one in-flight per ID” — the AXI completion is strictly in-order per ID, but the issue order at the scheduler is still OoO.

## 7.6 Timing Budget

| Path                                                | Budget                   |
|-----------------------------------------------------|--------------------------|
| awvalid $\rightarrow$ awready (combinational)       | $\leq$ 0.7 of cycle      |
| aw_buf push to addr_mapper valid                    | 0 cycles (combinational) |
| addr_mapper valid to CAM push                       | 1 cycle                  |
| wr_cmd_cam push to scheduler-visible                | 1 cycle                  |
| Worst path: awvalid $\rightarrow$ CAM-push register | 2 cycles                 |

The two-cycle AXI $\rightarrow$ CAM latency is the limit on AXI sustained issue rate; with default config it allows 1 burst per 2 cycles, well within the AXI burst rate at 200 MHz.

## 7.7 CSR Hooks

---

- `STATUS.axi_inflight_rd` (R only) — current count of in-flight read bursts
- `STATUS.axi_inflight_wr` (R only) — current count of in-flight write bursts
- `STATUS.axi_id_table_occ` (R only) — number of distinct AXI IDs currently in the side table
- `OBS_AXI_R_LATENCY_AVG`, `OBS_AXI_W_LATENCY_AVG` — driven from R/B-channel sampling in this FUB

## 7.8 TODO (week-of-MAS work)

---

- FSM diagram for the AW/W join and the AR intake (mermaid)
- B/R response-ordering corner cases (per HAS §3.1 OoO-across-IDs)
- Waveform examples (wavedrom) for back-pressure scenarios
- Detailed table of `id_side_table` field widths and reset values

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

# 8 Address XOR / Hash (`addr_mapper_fub`)

**Module:** `addr_mapper_fub.sv` **Location:** `rtl/fub/` **Category:** FUB (combinational; no clock)  
**Parent:** `ddr2_lpddr2_ctrl` **Status:** Skeleton

---

## 8.1 Purpose

---

`addr_mapper_fub` decodes a flat `AXI_ADDR_WIDTH`-bit address into the DRAM-layer (rank, bank, row, column) tuple per the currently-active address-mapping scheme. The XOR / hash happens **here** — upstream of the read/write CAMs — so that the CAMs store the decoded tuple and not the raw AXI address.

The FUB is **pure combinational** — there is no clock, no flop, no FSM. The output is valid the same cycle as the input. The path is one of the timing-critical signatures in the design and is treated as a multi-cycle path only if synthesis cannot meet timing.

## 8.2 Mirror to the Python AddressMapping Class

This RTL is the mirror of the AddressMapping Python class in the DV repository (CocoTBFramework.components.dfi.address\_mapping). Both produce **bit-for-bit identical** decoded outputs for the same input. The mirror is enforced by:

- Same scheme list (ROW\_MAJOR, BANK\_INTERLEAVE, XOR\_HASH)
- Same field-bit positions per scheme
- Same XOR\_HASH seed and tap matrix

The shared decode function eliminates a common class of bugs where simulation passes (because the BFM decodes one way) but RTL fails (because the controller decodes a different way).

## 8.3 Interface

| Signal          | Direction | Width        | Description                                                     |
|-----------------|-----------|--------------|-----------------------------------------------------------------|
| axi_addr_i      | input     | AW           | Flat AXI byte address                                           |
| scheme_active_i | input     | 2            | Runtime scheme select from ADDR_MAP_TUNING.scheme_or            |
| xor_seed_i      | input     | 8            | Runtime seed for XOR_HASH (from CSR)                            |
| bg_field_pos_i  | input     | 3            | Bank-group field bit position (unused in DDR2/LPDDR2; reserved) |
| rank_o          | output    | $\log_2(NR)$ | Decoded rank                                                    |
| bank_o          | output    | BA           | Decoded bank                                                    |
| row_o           | output    | ROW_WIDTH    | Decoded row                                                     |
| col_o           | output    | COL_WIDTH    | Decoded column (byte-aligned within burst)                      |

## 8.4 Scheme Decode

### 8.4.1 Scheme 1: ROW\_MAJOR

The simplest mapping; one contiguous row per rank/bank stride. Bit layout:

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | bank [BA-1:0] | col  
[COL_WIDTH-1:0] | byte_in_dq |
```

The byte-in-DQ field is the controller's column-byte offset and is consumed by wr\_data\_path\_fub for wstrb masking; it is **not** part of the decoded col\_o (which is column-word aligned).

ROW\_MAJOR is best for sequential streaming workloads where a long burst stays in one row.

#### 8.4.2 Scheme 2: BANK\_INTERLEAVE

```
| ... reserved ... | rank | row [ROW_WIDTH-1:0] | col_hi [...] | bank
[BA-1:0] | col_lo [...] | byte_in_dq |
```

Bank bits are placed **between** column-low and column-high bits, so consecutive cache lines round-robin across banks. This is the recommended default for general-purpose workloads with mixed access patterns; it produces higher bank-parallelism in the scheduler.

#### 8.4.3 Scheme 3: XOR\_HASH

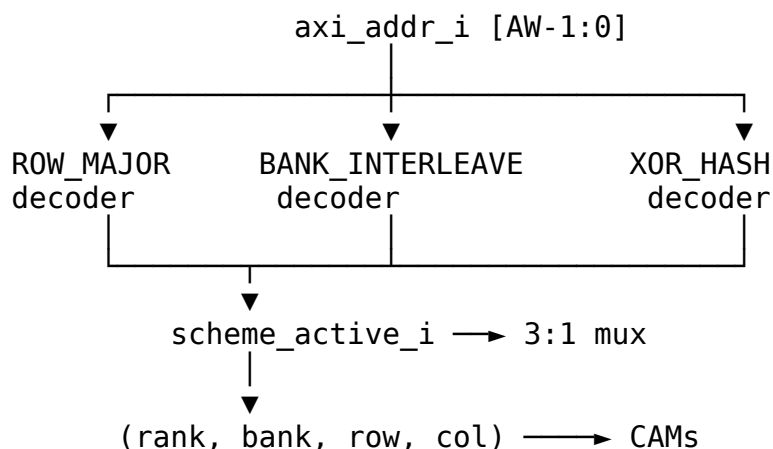
Same layout as BANK\_INTERLEAVE, but the bank field and (a subset of) the column-high bits are XOR-hashed with row bits:

```
bank_hashed[i] = bank_raw[i] XOR row_raw[i] XOR row_raw[i + BA] XOR
xor_seed[i]
```

The XOR\_HASH scheme defeats pathological row-conflict patterns (e.g., a power-of-two stride that lands on the same bank in BANK\_INTERLEAVE). The seed `xor_seed_i` is runtime-modifiable so a bring-up engineer can change the hash without rebuilding.

### 8.5 Runtime Scheme Mux

The output is multiplexed across the synthesized schemes per `scheme_active_i`. Only schemes in ADDR\_MAP\_SCHEMES\_SYNTH are routed to the mux; un-synthesized schemes tie off and writing them via CSR returns `pslverr`.



### 8.6 Timing Budget

The decoder is purely combinational. The longest path in XOR\_HASH is the XOR of (a) several address bits, (b) the seed, and (c) the row bits — about 5–6 levels of XOR gates plus a 3:1 mux. At 500 MHz this is comfortably within 0.5 ns of MC clock cycle time.



| Path                                     | Estimate (gate-level) |
|------------------------------------------|-----------------------|
| axi_addr_i → bank_o<br>(ROW_MAJOR)       | 2 levels of MUX       |
| axi_addr_i → bank_o<br>(BANK_INTERLEAVE) | 2 levels of MUX       |
| axi_addr_i → bank_o<br>(XOR_HASH)        | 5 levels XOR + 2 MUX  |
| axi_addr_i → row_o                       | 1 level MUX           |

If XOR\_HASH does not meet timing at the target frequency, the FUB can be re-fitted with a single pipeline stage (the seed-XOR can absorb a flop without changing the per-tuple invariant — the seed only changes on quiet points anyway).

## 8.7 Verification

The RTL decoder and the Python AddressMapping class are co-validated by a small cocotb test that picks 10000 random addresses, asks each side to decode, and asserts bit-equality on the (rank, bank, row, col) tuple. This is the primary regression for addr\_mapper\_fub.

## 8.8 TODO

- Tap matrix for XOR\_HASH — the exact bit mapping
- Synthesis hint for the XOR\_HASH critical path
- Edge cases for  $AW < (ROW\_WIDTH + COL\_WIDTH + BA + c\log_2(NR))$  (under-decoded address — should it tie-off or error?)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

# 9 Read Command CAM (rd\_cmd\_cam\_fub)

**Module:** rd\_cmd\_cam\_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2\_lpddr2\_ctrl  
**Status:** Skeleton

## 9.1 Purpose

`rd_cmd_cam_fub` is the **read-side content-addressable storage** of in-flight read commands. It holds the DRAM-layer view (rank, bank, row, col) of every read burst that has been pushed by `axi4_slave_fub` and not yet fully returned on the AXI R-channel.

The CAM is keyed by **AXI ID**, allowing the scheduler to find all reads to a given (rank, bank) without iterating the queue and allowing `rd_data_path_fub` to find the right entry to decrement when a read beat returns.

The read CAM and write CAM are intentionally separate FUBs — see §1.1 for the rationale.

## 9.2 Parameters

| Parameter                    | Default                   | Purpose                        |
|------------------------------|---------------------------|--------------------------------|
| <code>RD_CAM_DEPTH</code>    | 16                        | Number of in-flight read slots |
| <code>IDW</code>             | <code>AXI_ID_WIDTH</code> | Key width                      |
| <code>BURST_LEN_WIDTH</code> | 8                         | Burst-length field width       |

## 9.3 Storage Per Slot

| Field                       | Width                        | Description                                       |
|-----------------------------|------------------------------|---------------------------------------------------|
| <code>valid</code>          | 1                            | Slot occupied                                     |
| <code>axi_id</code>         | <code>IDW</code>             | Key — the originating AXI ID                      |
| <code>rank</code>           | $\$clog2(NR)$                | Decoded rank                                      |
| <code>bank</code>           | <code>BA</code>              | Decoded bank                                      |
| <code>row</code>            | <code>ROW_WIDTH</code>       | Decoded row                                       |
| <code>col_start</code>      | <code>COL_WIDTH</code>       | Starting column                                   |
| <code>burst_len</code>      | <code>BURST_LEN_WIDTH</code> | Total beats in burst                              |
| <code>beats_returned</code> | <code>BURST_LEN_WIDTH</code> | Beats already returned on R                       |
| <code>issued</code>         | 1                            | Scheduler has picked this entry                   |
| <code>last_beat_at</code>   | small                        | Cycle of last beat return (for latency telemetry) |

Total per-slot width (default config): ~46 bits.

Total storage (16 entries × 46 b): ~92 bytes — small enough to fit comfortably in distributed flops with no SRAM macro.

## 9.4 Interfaces

### 9.4.1 Push (from axi4\_slave\_fub)

| Signal     | Direction | Width                      | Description             |
|------------|-----------|----------------------------|-------------------------|
| push_valid | input     | 1                          | New read burst arriving |
| push_ready | output    | 1                          | CAM has a free slot     |
| push_id    | input     | IDW                        | AXI ID                  |
| push_rank  | input     | $\lceil \log_2(NR) \rceil$ | from addr_mapper        |
| push_bank  | input     | BA                         |                         |
| push_row   | input     | ROW_WIDTH                  |                         |
| push_col   | input     | COL_WIDTH                  |                         |
| push_len   | input     | BURST_LEN_WIDTH            |                         |

### 9.4.2 Scheduler Query

The scheduler queries the CAM to:

- Find all unissued entries matching a target (rank, bank) — for next-issue selection
- Find row-hit candidates (entries whose row == open\_row[rank][bank])

| Signal           | Direction | Description                                                       |
|------------------|-----------|-------------------------------------------------------------------|
| q_rank_i         | input     | Query rank                                                        |
| q_bank_i         | input     | Query bank                                                        |
| q_row_i          | input     | Query row (for row-hit check)                                     |
| match_unissued_o | output    | One-hot vector of slots matching (rank, bank) and not issued      |
| match_rowhit_o   | output    | One-hot vector of slots matching (rank, bank, row) and not issued |
| match_data_o     | output    | The selected slot's payload (rank, bank, row, col, len)           |

### 9.4.3 Issue Notification

When the scheduler picks an entry from this CAM:

| Signal        | Direction | Description                  |
|---------------|-----------|------------------------------|
| issued_slot_i | input     | Slot index to mark as issued |
| issued_we_i   | input     | Mark-issued strobe           |

#### 9.4.4 Beat Return (from rd\_data\_path\_fub)

| Signal           | Direction | Description                                                            |
|------------------|-----------|------------------------------------------------------------------------|
| beat_id_i        | input     | AXI ID of returning beat                                               |
| beat_we_i        | input     | Beat-arrived strobe                                                    |
| entry_complete_o | output    | One-hot vector of slots that just completed (last beat of their burst) |

On entry-complete the slot's valid is cleared in the next cycle.

## 9.5 Lookup Mechanism

The CAM uses a **parallel match-line** structure: each slot has a small comparator for {rank, bank} that drives a per-slot match wire. The scheduler reads the full match-vector and picks among matches using a priority encoder (age-priority — oldest slot wins ties).

For the typical RD\_CAM\_DEPTH of 16 with BA=3,  $\lceil \log_2(NR) \rceil = 1..2$  (1 for single-rank, 2 for quad-rank), this is 16 4-5-bit equality comparators in parallel — trivial silicon.

## 9.6 Per-ID Beat Counting

Because reads return in-order *per AXI ID* (an AXI ordering rule), the read CAM does not need a per-beat slot table. A slot's beats\_returned increments on each beat that returns with that ID, and when beats\_returned == burst\_len the slot completes and clears.

If AXI\_000\_ACROSS\_IDS == false (rare), the SoC has asked for in-order return across all IDs, and the read CAM enforces this by issuing a head-of-queue priority hint to the scheduler — entries are still cleared in arrival order. The OoO-across-IDs default is true; per-ID in-order is the AXI default and is preserved.

## 9.7 Timing Budget

| Path                               | Budget        |
|------------------------------------|---------------|
| push_valid → slot occupy (1 cycle) | 1 cycle       |
| q_rank_i / q_bank_i → match*_o     | combinational |
| beat_id_i →                        | 1 cycle       |

| Path                          | Budget  |
|-------------------------------|---------|
| beats_returned[slot]+1        |         |
| entry_complete_o → slot clear | 1 cycle |

The combinational match path is on the scheduler's critical loop and is shared with `wr_cmd_cam_fub` and the bank machines' state matrix. See `ch02_blocks/07_scheduler.md` for the full critical-path breakdown.

## 9.8 CSR Hooks

- `STATUS.rd_cam_occ` — current occupancy
- `OBS_RD_CAM_HIGH_WATER` — peak occupancy observed since last read

## 9.9 TODO

- Mermaid for the parallel match-line topology
- Wavedrom: push → scheduler-match → issue → beats return → clear
- Per-ID head-of-queue ordering corner case spec

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

# 10 Write Command CAM (`wr_cmd_cam_fub`)

**Module:** `wr_cmd_cam_fub.sv` **Location:** `rtl/fub/` **Category:** FUB **Parent:** `ddr2_lpddr2_ctrl`  
**Status:** Skeleton

## 10.1 Purpose

`wr_cmd_cam_fub` is the **write-side content-addressable storage** of in-flight write commands. It mirrors the read CAM's role for write traffic but carries different per-slot metadata:

- Pointer into the AXI-side write-data buffer (`w_buf_ptr`)
- Pointer into the AXI-side write-strobe buffer (`strb_ptr`)
- Per-slot beats-issued counter (so the FUB knows when the last W beat has been pushed to DFI write data)

Separation from the read CAM lets the write path sit shallow — writes complete on B response, which is a single AXI event, so the write CAM does not need to track per-beat return.

## 10.2 Parameters

| Parameter       | Default                  | Purpose                         |
|-----------------|--------------------------|---------------------------------|
| WR_CAM_DEPTH    | 16                       | Number of in-flight write slots |
| IDW             | AXI_ID_WIDTH             | Key width                       |
| W_BUF_PTR_WIDTH | $\$clog2(W\_BUF\_DEPTH)$ | Pointer into AXI W buffer       |
| BURST_LEN_WIDTH | 8                        | Burst-length field width        |

## 10.3 Storage Per Slot

| Field        | Width           | Description                                   |
|--------------|-----------------|-----------------------------------------------|
| valid        | 1               | Slot occupied                                 |
| axi_id       | IDW             | Key — the originating AXI ID                  |
| rank         | $\$clog2(NR)$   | Decoded rank                                  |
| bank         | BA              | Decoded bank                                  |
| row          | ROW_WIDTH       | Decoded row                                   |
| col_start    | COL_WIDTH       | Starting column                               |
| burst_len    | BURST_LEN_WIDTH | Total beats                                   |
| beats_issued | BURST_LEN_WIDTH | Beats already pushed to DFI write data        |
| w_buf_ptr    | W_BUF_PTR_WIDTH | Pointer into the AXI W-buffer for next beat   |
| strb_ptr     | W_BUF_PTR_WIDTH | Pointer into the per-beat strobe buffer       |
| issued       | 1               | Scheduler has picked this entry               |
| b_pending    | 1               | Awaiting B response from DFI write completion |

Total per-slot width: ~58 bits in the default config. Total storage ( $16 \times 58$ ): ~116 bytes, distributed flops.

## 10.4 Interfaces

### 10.4.1 Push (from axi4\_slave\_fub)

Same shape as the read CAM push, plus `w_buf_ptr` and `strb_ptr` (which the AXI slave allocates at push time).

### 10.4.2 Scheduler Query

Same shape as the read CAM. The scheduler matches by (rank, bank) and selects with row-hit priority.

### 10.4.3 Issue Notification

When the scheduler picks a write, the CAM: 1. Marks `issued = 1` 2. Begins handing per-beat data to `wr_data_path_fub` via the `beat_data_pull` interface (below)

### 10.4.4 Beat-Pull (to wr\_data\_path\_fub)

| Signal                        | Direction | Description                           |
|-------------------------------|-----------|---------------------------------------|
| <code>beat_pull_o</code>      | output    | Strobe — pull next beat for this slot |
| <code>beat_pull_slot_o</code> | output    | Slot index whose beat is being pulled |
| <code>beat_pull_ptr_o</code>  | output    | <code>w_buf_ptr + beats_issued</code> |
| <code>beat_pull_strb_o</code> | output    | <code>strb_ptr + beats_issued</code>  |
| <code>beat_pull_last_o</code> | output    | 1 when this is the burst's final beat |

`wr_data_path_fub` walks the AXI W buffer at the supplied pointer and pushes to DFI `wrdata`. On the last beat, it asserts `b_pending_set` to the CAM.

### 10.4.5 Completion (from DFI write window expiration)

| Signal                        | Direction | Description                                                                    |
|-------------------------------|-----------|--------------------------------------------------------------------------------|
| <code>b_pending_set_i</code>  | input     | Last W beat pushed; await <code>tCWL+tWR</code> window                         |
| <code>b_complete_i</code>     | input     | Write window expired (from <code>xbank_timers</code> )                         |
| <code>entry_complete_o</code> | output    | Burst is fully complete; signal <code>axi4_slave_fub</code> to push B response |

## 10.5 Difference From Read CAM

| Aspect    | Read CAM                            | Write CAM                                                |
|-----------|-------------------------------------|----------------------------------------------------------|
| Lifecycle | Push → issue → beats return → clear | Push → issue → beats issued → write window expires → B → |

| Aspect            | Read CAM                              | Write CAM                                                                  |
|-------------------|---------------------------------------|----------------------------------------------------------------------------|
|                   |                                       | clear                                                                      |
| Beat tracking     | Counts beats <i>returned</i> from DFI | Counts beats <i>issued</i> to DFI; also tracks the post-issue write window |
| AXI ID echo       | rid returned with each R beat         | bid returned once at B-response time                                       |
| Pointer fields    | none (just a beat counter)            | w_buf_ptr, strb_ptr into AXI-side buffers                                  |
| Completion driver | rd_data_path (last beat of R)         | xbank_timers (write window expiration) → wr_data_path                      |

The asymmetry justifies the separate FUBs. Trying to share a CAM would either inflate the read slot to carry write-only fields, or vice versa.

## 10.6 Timing Budget

Same shape as the read CAM. The combinational match-line path is shared with rd\_cmd\_cam\_fub in the scheduler's match aggregation.

## 10.7 CSR Hooks

- STATUS.wr\_cam\_occ — current occupancy
- STATUS.wr\_b\_pending — count of slots awaiting B response
- OBS\_WR\_CAM\_HIGH\_WATER — peak occupancy

## 10.8 TODO

- Mermaid for the issued → beat-pull → b\_pending → b\_complete lifecycle
- Wavedrom showing a multi-beat write burst from CAM push to B response
- Backpressure path when wr\_data\_path can't keep up with beat\_pull (rare; DFI wrdata is wider than AXI W)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)



## 11 Transaction Queue (txn\_queue\_fub)

**Module:** txn\_queue\_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2\_lpddr2\_ctrl  
**Status:** Draft v0.1

Architectural context: HAS §3.2 txn\_queue. This block-level MAS section is the implementation detail: entry format, port arrangement, insert/clear paths, row\_hit\_cached coherency, backpressure, storage choice.

---

### 11.1 Purpose

---

txn\_queue\_fub is the **scheduling view** of in-flight DRAM transactions. It is the data structure the scheduler reads every MC clock to pick the next command. The queue is not the source of truth for AXI metadata or beat tracking — that lives in rd\_cmd\_cam\_fub and wr\_cmd\_cam\_fub. The queue carries *only* the fields the scheduler’s priority function needs, plus a reverse pointer back to the CAM slot.

The split exists because the scheduler’s read path is the hottest in the design: every cycle it touches every entry in parallel. Making the entry narrow (~32 bits, see below) keeps the parallel-comparator fan-in manageable. The CAMs carry the wider per-burst metadata (w\_buf\_ptr, strb\_ptr, beats\_returned, etc.) which is only touched on insert, on scheduler pick, and on per-beat completion — never in the per-cycle scheduling loop.

---

### 11.2 Relationship to the CAMs

---

Each entry in txn\_queue reflects exactly one slot in rd\_cmd\_cam or wr\_cmd\_cam. The cam\_slot\_idx field is the reverse pointer:

txn\_queue[q].cam\_slot\_idx → (is\_write ? wr\_cmd\_cam[cam\_slot\_idx] : rd\_cmd\_cam[cam\_slot\_idx])

When the scheduler picks txn\_queue[q], it issues an “mark issued” strobe to the corresponding CAM using cam\_slot\_idx (see scheduler\_fub, §2.7). When the CAM signals entry-complete (read: last beat returned; write: B-channel quiet point reached), it strobes back to clear txn\_queue[q].

The queue and CAMs are sized independently:

| Structure | Default depth | Reason                                       |
|-----------|---------------|----------------------------------------------|
| txn_queue | 16            | Scheduling pool — depth bounded by parallel- |

| Structure  | Default depth | Reason                     |
|------------|---------------|----------------------------|
|            |               | comparator cost            |
| rd_cmd_cam | 16            | One slot per pending read  |
| wr_cmd_cam | 16            | One slot per pending write |

In the default 16+16+16 configuration, the queue is the bottleneck — at most 16 transactions can be queued for scheduling at any time, regardless of how many slots are free in each CAM individually. The CAMs are not allowed to oversubscribe the queue (the axi4\_slave\_fub push path stalls on txn\_queue full, not on CAM-full). The two CAMs *can* be sized larger than the queue if the read-path and write-path have very different burst-completion latencies — but that's not v1.

## 11.3 Synthesis Parameters

| Parameter       | Source            | Effect                                                                      |
|-----------------|-------------------|-----------------------------------------------------------------------------|
| TXN_QUEUE_DEPTH | top (default 16)  | Number of parallel-readable entries                                         |
| AGE_MAX         | top (default 256) | Age counter saturation; per-entry $\text{clog}_2(\text{AGE\_MAX}) = 8$ bits |
| ROW_WIDTH       | top               | Width of the row field                                                      |
| NUM_RANKS       | top               | Width of the rank field                                                     |
| NUM_BANKS       | top               | Width of the bank field                                                     |

## 11.4 Entry Format

Each slot carries the **scheduling view** — the minimum metadata the scheduler's priority function needs.

| Field    | Width                              | Description                                                  |
|----------|------------------------------------|--------------------------------------------------------------|
| valid    | 1                                  | Slot occupied                                                |
| state    | 2                                  | {FREE, PENDING, ISSUED, COMPLETING}<br>(see lifecycle below) |
| is_write | 1                                  | Picks rd vs wr CAM for the reverse pointer                   |
| rank     | $\text{clog}_2(\text{NUM\_RANKS})$ | Target rank (1-2 bits)                                       |
| bank     | $\text{clog}_2(\text{NUM\_BANKS})$ | Target bank (3 bits for NB=8)                                |

| Field                 | Width                 | Description                                               |
|-----------------------|-----------------------|-----------------------------------------------------------|
| row                   | ROW_WIDTH             | Target row (14 bits default)                              |
| row_hit_cached        | 1                     | Cached at insert; refreshed on bank-state change          |
| age                   | $\$clog2(AGE\_MAX)$   | Saturating age counter (8 bits default)                   |
| cam_slot_idx          | $\$clog2(CAM\_DEPTH)$ | Reverse pointer to rd_cmd_cam or wr_cmd_cam slot (4 bits) |
| <b>Total per slot</b> | <b>~32 bits</b>       | (default: NR=1, NB=8, ROW=14, AGE_MAX=256, CAM=16)        |

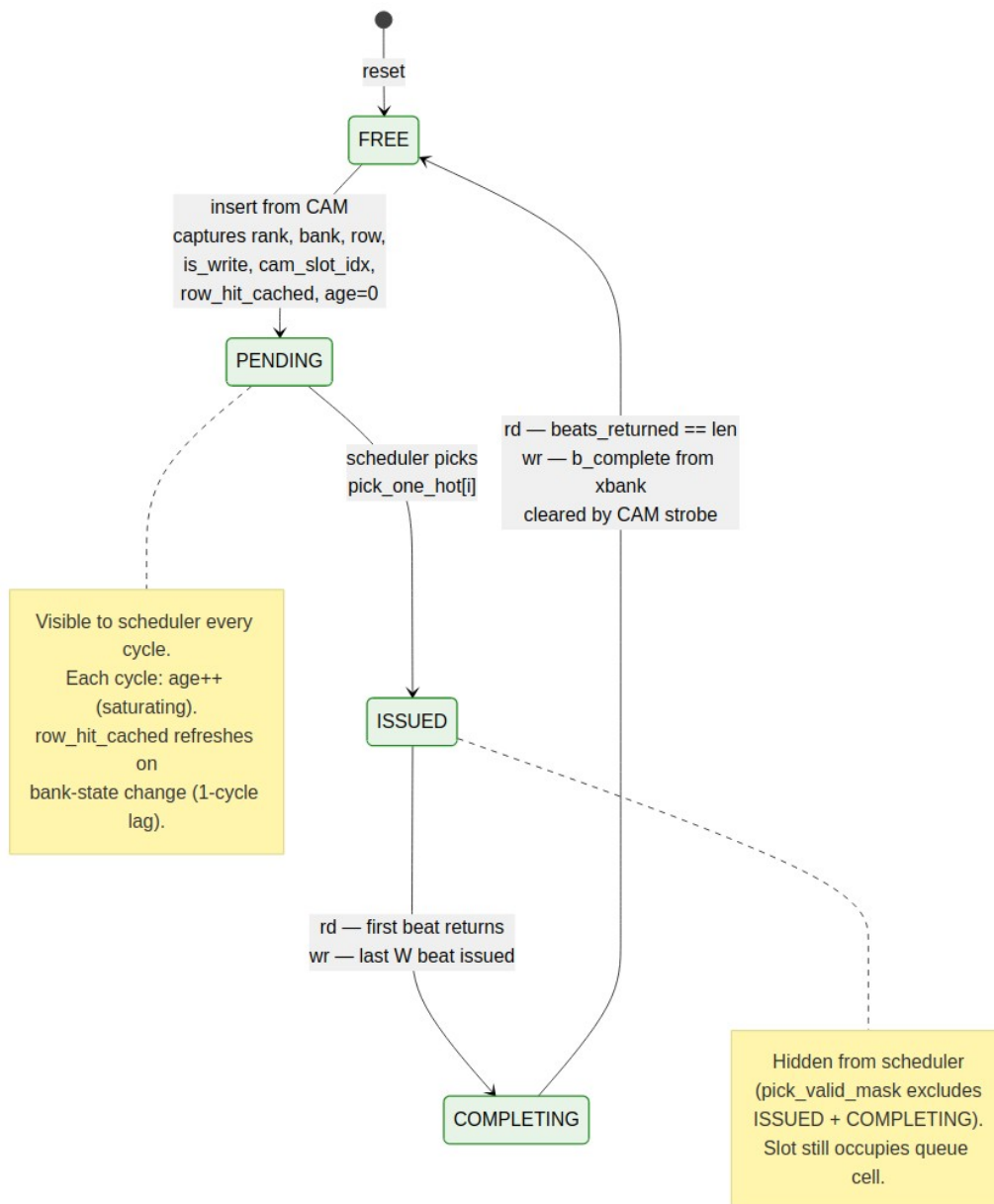
Total queue storage at depth 16: **~64 bytes** ( $16 \times 32$  b). This is distributed flops — no SRAM macro. Even at depth 64 it's 256 bytes, still trivially distributed.

**Fields *not* in the queue entry** (these stay in the CAM):

- col\_start, burst\_len — only consulted at issue time (Stage 4 of scheduler), not every cycle
- axi\_id — opaque to the scheduler; only the CAM and axi4\_slave\_fub need it
- w\_buf\_ptr, strb\_ptr, beats\_returned, beats\_issued — beat-level tracking, not scheduling

At issue time, the scheduler reads col\_start and burst\_len via the cam\_slot\_idx reverse pointer — a single read from the CAM's lookup port. This is one cycle slower than carrying them in the queue, but it keeps the queue narrow.

## 11.5 Entry Lifecycle



### *txn\_queue Entry State Diagram*

**Source:** [06\\_txn\\_queue\\_entry\\_state.mmd](#)

Four states tracked per slot:

| State      | Visible to scheduler? | Counts toward queue depth? | Notes                                         |
|------------|-----------------------|----------------------------|-----------------------------------------------|
| FREE       | No                    | No                         | Empty slot, available for insert              |
| PENDING    | Yes (eligible)        | Yes                        | Scheduler is searching for this entry         |
| ISSUED     | No (masked out)       | Yes                        | Picked by scheduler; column command in flight |
| COMPLETING | No (masked out)       | Yes                        | Last beat / write-window draining             |

The scheduler's `pick_valid_mask` (see §2.7 Stage 2) explicitly excludes ISSUED and COMPLETING so the same entry can never be picked twice. The mask is `(state == PENDING)`.

ISSUED → COMPLETING is the same edge in both rd and wr cases but triggered by different signals:

- **Read:** first R beat returns from DFI → `rd_data_path_fub` strobes `entry_transition_completing` via the CAM.
- **Write:** last W beat is pushed to DFI write data → `wr_data_path_fub` strobes the same edge.

The COMPLETING → FREE edge is driven by the CAM's `entry_complete_o` strobe (see §2.4 and §2.5). The queue does not own completion-detection logic; it follows the CAM.

## 11.6 Ports

### 11.6.1 Insert (from CAMs)

| Signal                      | Direction | Width                 | Description                   |
|-----------------------------|-----------|-----------------------|-------------------------------|
| <code>ins_valid_i</code>    | input     | 1                     | New entry to push             |
| <code>ins_ready_o</code>    | output    | 1                     | Queue has a free slot         |
| <code>ins_is_write_i</code> | input     | 1                     |                               |
| <code>ins_rank_i</code>     | input     | $\$clog2(NR)$         | from <code>addr_mapper</code> |
| <code>ins_bank_i</code>     | input     | $\$clog2(NB)$         |                               |
| <code>ins_row_i</code>      | input     | ROW_WIDTH             |                               |
| <code>ins_cam_slot_i</code> | input     | $\$clog2(CAM\_DEPTH)$ | Reverse pointer               |

row\_hit\_cached is **not** an insert port — it's computed inside this FUB at insert time by querying bank\_state\_i[ins\_rank][ins\_bank].open\_row (see “Insert Path” below). This keeps the insert-time row-hit computation local to the queue.

### 11.6.2 Parallel Read (to scheduler — every cycle)

| Signal                           | Direction | Width                  | Description                          |
|----------------------------------|-----------|------------------------|--------------------------------------|
| q_entries_o[TXN_QUEUE_DEPTH-1:0] | output    | per entry-format table | Live snapshot of all slots, parallel |

This is a wide bus — at the default TXN\_QUEUE\_DEPTH = 16 and ~32 bits per entry, it's a 512-bit bus. The scheduler reads it combinatorially; there's no read-port arbitration because there's only one reader.

### 11.6.3 Bank-State Watchpoint (from bank machines)

| Signal                          | Direction | Width             | Description                                          |
|---------------------------------|-----------|-------------------|------------------------------------------------------|
| bank_state_i[NR][NB]            | input     | per bank_machine  | Live bank state (used at insert for row_hit_cached)  |
| bank_open_row_changed_i[NR][NB] | input     | NR×NB             | One-cycle strobe when a bank's open row changes      |
| bank_new_open_row_i[NR][NB]     | input     | NR×NB × ROW_WIDTH | New open-row value, valid the cycle after the strobe |

The \_changed + \_new\_open\_row pair is what drives the per-entry row\_hit\_cached refresh path (see “Row-Hit Cache Coherency” below).

### 11.6.4 Scheduler Pick (from scheduler)

| Signal         | Direction | Width           | Description                                       |
|----------------|-----------|-----------------|---------------------------------------------------|
| pick_one_hot_i | input     | TXN_QUEUE_DEPTH | One-hot vector of which slot the scheduler picked |
| pick_strobe_i  | input     | 1               | Pick happened this cycle                          |

On pick\_strobe\_i, the indicated slot transitions PENDING → ISSUED.

### 11.6.5 CAM Completion (from CAMs)

| Signal                    | Direction | Width                                            | Description                                      |
|---------------------------|-----------|--------------------------------------------------|--------------------------------------------------|
| entry_transition_i        | input     | 2                                                | 01 = ISSUED → COMPLETING, 10 = COMPLETING → FREE |
| entry_transition_slot_i   | input     | $\lceil \log_2(\text{TXN\_QUEUE\_DEPTH}) \rceil$ | Slot index that's transitioning                  |
| entry_transition_strobe_i | input     | 1                                                | Transition strobe                                |

### 11.6.6 Backpressure Output

| Signal         | Direction | Width                                    | Description                                                  |
|----------------|-----------|------------------------------------------|--------------------------------------------------------------|
| q_high_water_o | output    | 1                                        | High when occupancy $\geq$ SCHED_TUNING.txn_queue_high_water |
| q_full_o       | output    | 1                                        | All slots are non-FREE                                       |
| q_occupancy_o  | output    | $\lceil \log_2(\text{DEPTH} + 1) \rceil$ | Current occupancy (number of slots not in FREE state)        |

q\_high\_water\_o drives the AXI backpressure path in axi4\_slave\_fub — awready / arready deasserts when the queue is near full. q\_full\_o is the hard backpressure (no inserts accepted).

## 11.7 Insert Path

When axi4\_slave\_fub has decoded a new burst and pushed it to either CAM, the CAM in turn pushes to this queue:

```

cycle T:  CAM push from axi4_slave → ins_valid asserted, ins_ready
asserted
cycle T:  queue selects free_slot via priority encoder over (state ==
FREE)
cycle T:  row_hit_cached_init = (bank_state[ins_rank][ins_bank].state
== ACTIVE
                                AND bank_state[ins_rank]
[ins_bank].open_row == ins_row)
cycle T+1: free_slot.{is_write, rank, bank, row, row_hit_cached,

```

```
cam_slot_idx} latched
cycle T+1: free_slot.state = PENDING, age = 0
```

The insert is a **two-cycle latency** from CAM push to first scheduler visibility, but the queue can accept one new insert per cycle (the comparator network for `free_slot` finds the next available slot combinationally; the latch is registered).

`row_hit_cached_init` is the only nontrivial computation at insert time. It's a single comparison: does the target bank's open row match the inserted row? If the bank is IDLE, it's automatically 0 (no row is open). If the bank is in the middle of a row change (ACTIVATING or PRECHARGING), it's also 0 — the cached value will be refreshed in the next cycle when the bank settles via the watchpoint path below.

---

## 11.8 Row-Hit Cache Coherency

---

The `row_hit_cached` field is computed at insert (above) and **refreshed** any time a bank's open row changes.

The refresh path:

```
cycle T:   bank_machine[r,b] transitions ACTIVE → ACTIVE-on-new-row
           (via PRE then ACT) OR IDLE → ACTIVE (via ACT)
cycle T:   bank_machine[r,b] strobes bank_open_row_changed[r][b] = 1
           and presents bank_new_open_row[r][b] = R_new
cycle T+1: txn_queue refresh path:
           for each slot i:
               if entries[i].state in {PENDING}
                   AND entries[i].rank == r AND entries[i].bank == b:
                       entries[i].row_hit_cached = (entries[i].row == R_new)
```

The refresh is broadcast-parallel: all `TXN_QUEUE_DEPTH` entries are checked in parallel (their (rank, bank) matches are computed; the matching set has its `row_hit_cached` recomputed). The refresh fires for ISSUED and COMPLETING entries too, but those are masked out by the scheduler anyway — refreshing them is harmless and cheaper than gating.

**Why a one-cycle lag is acceptable.** In the cycle the bank state changes, the scheduler may still observe stale `row_hit_cached`. The worst outcome is the scheduler picks an entry as a “row hit” that isn't, or vice versa. The bank\_machine catches this — `accepts_rd / accepts_wr` are gated on the current `open_row`, so the scheduler can't issue an *incorrect* column command. It would issue a bare RD/WR when an RDA/WRA was warranted (or vice versa), costing at most one extra PRE later. This is the trade documented in scheduler §2.7.

The synchronous broadcast (rather than asynchronous combinational re-eval) keeps the scheduler's Stage-1 path off the bank\_machine state machines. If we tried to make this combinational, the Stage-1 critical path would route through every bank machine's `open_row`



register, every queue entry's row comparator, and every priority mask — easily 8-10 LUT levels, ruining the timing budget.

---

## 11.9 Age Counter Saturation

---

Every cycle, every PENDING entry's age increments by 1. The counter saturates at  $\text{AGE\_MAX} - 1$ . Concretely with  $\text{AGE\_MAX} = 256$ :

- New entry: age = 0
- After 1 cycle: age = 1
- After 255 cycles: age = 255
- After 256+ cycles: age = 255 (saturated)

The scheduler's age priority encoder treats saturated entries equally — once an entry has been waiting “long enough,” it has maximum priority and ties break by slot index. In normal traffic, ages rarely saturate; in pathological cases (a row-conflict victim behind a stream of row-hits) saturation provides the anti-starvation guarantee.

The runtime override `SCHED_TUNING.age_max_runtime` clips the effective saturation point lower than the build-time  $\text{AGE\_MAX}$  — useful for characterization sweeps to shorten the saturation timescale without rebuilding.

**Increment gating.** Age does *not* increment when state is ISSUED or COMPLETING. Those entries are out of the scheduling pool; aging them further would skew the priority for any subsequent entries that get re-queued (which doesn't currently happen in v1, but the gate is cheap insurance).

---

## 11.10 Storage Choice

---

At the default  $\text{TXN\_QUEUE\_DEPTH} = 16$ , total storage is ~64 bytes. Distributed flops are the right answer — an SRAM macro would have higher access latency and the macro overhead is significant for so little data.

The crossover point where SRAM becomes attractive is around  $\text{TXN\_QUEUE\_DEPTH} \geq 64$  (256 bytes), and even then only if the scheduler can be re-architected to issue from a partial snapshot rather than the full queue. Currently, the scheduler reads all entries in parallel — SRAM ports can't deliver that. So in v1 we cap at distributed flops; the parameter range is 4 ... 64.

For depth = 64, the entry bus is  $64 \times 32 = 2048$  bits — wide but routable on 7-series. Stage-1 priority comparators scale linearly with depth (16 → 64 is 4× the comparators), and the age priority encoder tournament grows by  $\log_2(4) = 2$  levels. The critical-path budget in §2.7 absorbs this at the 200 MHz target; at 500 MHz the scheduler will need flop insertion as noted.

---

## 11.11 CSR Hooks

| CSR field                               | Source signal                               | Use case                                  |
|-----------------------------------------|---------------------------------------------|-------------------------------------------|
| STATUS.txn_queue_overflow (R)           | q_occupancy_o                               | Live occupancy                            |
| OBS_TXN_QUEUE_DEPTH_MAX (R)             | High-water-mark register                    | Peak occupancy since last read (HAS §6.3) |
| OBS_TXN_QUEUE_DEPTH_AVG (R)             | Rolling time-average                        | Time-averaged occupancy                   |
| STATUS.txn_queue_full_pulses (R)        | Counter — number of times q_full_o asserted | Hard-backpressure event counter           |
| SCHED_TUNING.txn_queue_high_water (R/W) | Threshold for q_high_water_o                | Software-tunable soft-backpressure point  |

The `_high_water` threshold defaults to `TXN_QUEUE_DEPTH - 2` so AXI has a few-burst headroom before hard backpressure. Software can tune it lower for tighter latency control.

---

## 11.12 Verification Notes (cocotb test plan)

| Scenario                                                   | What it proves                                        |
|------------------------------------------------------------|-------------------------------------------------------|
| Single insert, single pick, single clear                   | Smoke: FREE → PENDING → ISSUED → COMPLETING → FREE    |
| Insert at full speed (1 per cycle) until full              | Insert throughput; q_full_o asserts on the 17th       |
| Bank-state change while N entries target that (rank, bank) | Broadcast row-hit refresh works on all matching slots |
| Bank-state change <i>during</i> a pick on the same slot    | Pick is honored; refresh applies to next cycle's view |
| Age saturation across all queue slots                      | All saturate at AGE_MAX - 1; tie-break by slot index  |
| age_max_runtime = 16, age saturates earlier                | Runtime clip works                                    |

| Scenario                                                                | What it proves                         |
|-------------------------------------------------------------------------|----------------------------------------|
| than AGE_MAX                                                            |                                        |
| Insert + pick + complete in the same cycle (different slots)            | The four ports are non-conflicting     |
| txn_queue_high_water = 8; AXI backpressure asserts at occupancy 8       | Soft backpressure threshold            |
| q_full_o blocks CAM push; CAM stalls AXI                                | Hard backpressure path                 |
| Insert with is_write = 0 and is_write = 1 interleaved                   | Reverse-pointer routing to correct CAM |
| entry_transition with slot out-of-range (illegal)                       | Assertion fires; entry untouched       |
| Reset during mid-life: all slots clear, no spurious completions emitted | Reset behavior                         |

### 11.13 Open Questions / Future Work

- Should the queue allow **re-queue** of a COMPLETING entry that fails (e.g., a DFI rddata timeout)? Currently the CAM owns failure-recovery; if it decides to retry, it would have to push a new queue entry. A “re-queue without losing age” path would be cheaper but adds complexity. Defer to v2.
- Should cam\_slot\_idx be **wider** than  $\$clog2(CAM\_DEPTH)$  to allow over-sized CAMs? Currently sized to match the smaller of the two CAM depths. If the rd-CAM grows to 32 and wr-CAM stays at 16, the field width adapts at elaboration; this is fine but the field is technically over-provisioned for write entries. Not worth fixing in v1.
- The broadcast row\_hit\_cached refresh fires for ISSUED / COMPLETING entries too. These are masked at the scheduler anyway. The gate would save a few comparator switches but no flops — drop the gate optimization. Document the decision and move on.

## 12 FR-FCFS Scheduler (scheduler\_fub)

**Module:** scheduler\_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2\_lpddr2\_ctrl  
**Status:** Draft v0.1

Architectural context: HAS §3.2 (scheduler, priority function, lookahead, force\_inorder, refresh-priority). This block-level MAS section is the implementation view: signals, internal datapath, pipeline staging, critical paths, and verification scenarios.

---

### 12.1 Purpose

---

scheduler\_fub is the central command issue engine. Every MC clock cycle it:

1. Snapshots txn\_queue and all bank\_machine states
2. Computes a per-entry readiness vector (which queue entries *could* issue right now)
3. Applies priority masking (init, refresh, force-inorder, row-hit-first, age tie-break)
4. Picks at most one winner via parallel priority encoders
5. Generates the DRAM command (issue\_op, operands) and the CAM “mark issued” strobe

Issue rate is **one command per MC clock**. Multi-cycle commands (LPDDR2’s 2-cycle CA encodings) are absorbed inside cmd\_encoder\_fub and gear\_dfi\_fub; the scheduler still sees one issue per cycle.

The block is the **single hardest synthesis-timing FUB** in the design — the parallel match-line into per-entry readiness, then into priority encoders, is what sets the MC clock frequency ceiling. Everything else has slack.

---

### 12.2 Synthesis Parameters

---

| Parameter      | Source | Effect on this FUB                                                                                                                                 |
|----------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| SCHEDULER_MODE | top    | "000" synthesizes the full FR-FCFS comparator network; "INORDER" synthesizes only the first-ready FIFO path (smaller area, no row-hit reordering). |

| Parameter               | Source            | Effect on this FUB                                        |
|-------------------------|-------------------|-----------------------------------------------------------|
| TXN_QUEUE_DEPTH         | top (default 16)  | Number of parallel readiness comparators                  |
| LOOKAHEAD_DEPTH_MAX     | top (default 4)   | Width of the lookahead window peek into txn_queue         |
| PAGE_POLICY             | top               | Picks which Stage-4 auto-precharge decoder is synthesized |
| NUM_RANKS,<br>NUM_BANKS | top               | Width of the bank-state input matrix                      |
| AGE_MAX                 | top (default 256) | Width of per-entry age counter (clog2(AGE_MAX))           |

The "INORDER" synthesis variant is a different decoder tree, not a runtime gate of the OoO tree. The runtime SCHED\_TUNING.force\_inorder bit is honored *only* when SCHEDULER\_MODE == "000" was synthesized (the bit becomes a tied observation in INORDER builds, per HAS §3.2).

## 12.3 Interface

### 12.3.1 Inputs

| Signal                             | Direction | Width                | Description                                                                                                                 |
|------------------------------------|-----------|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| mc_clk, mc_rst_n                   | input     | 1, 1                 | MC clock + async-assert / sync-deassert reset                                                                               |
| q_entries_i[TXN_QUEUE_DEPTH-1:0]   | input     | per HAS §3.2         | Live queue snapshot: per entry {valid, is_write, rank, bank, row, col, burst_len, row_hit_cached, age, state, cam_slot_idx} |
| bank_state_i[NUM_RANKS][NUM_BANKS] | input     | per bank_machine_fub | Per-(rank, bank): state, open_row, accepts_act/rd/wr/pre/ref                                                                |
| xbank_blocks_i                     | input     | struct               | blocks_act_per_rank[NR], blocks_xrank_rd, blocks_xrank_wr                                                                   |
| refresh_wants_i                    | input     | 1                    | One-bit "refresh due now" from refresh_mgr_fub                                                                              |

| Signal                                       | Direction | Width | Description                                                       |
|----------------------------------------------|-----------|-------|-------------------------------------------------------------------|
| refresh_done_for_i[<br>NUM_RANKS][NUM_BANKS] | input     | NR×NB | Per-(rank, bank) “refresh handshake granted” from refresh_mgr_fub |
| init_in_progress_i                           | input     | 1     | From init_engine_fub — blocks all normal traffic                  |
| predict_hit_i[<br>NUM_RANKS][NUM_BANKS]      | input     | NR×NB | HAPPY predictor output (tied 0 when PAGE_POLICY != HAPPY_HYBRID)  |
| cfg_force_inorder_i                          | input     | 1     | SCHED_TUNING.force_inorder runtime override                       |
| cfg_lookahead_active_i                       | input     | 4     | SCHED_TUNING.lookahead_active                                     |
| cfg_page_policy_or_i                         | input     | 2     | REFRESH_TUNING.page_policy_or                                     |
| cfg_happy_enable_i                           | input     | 1     | SCHED_TUNING.happy_enable                                         |

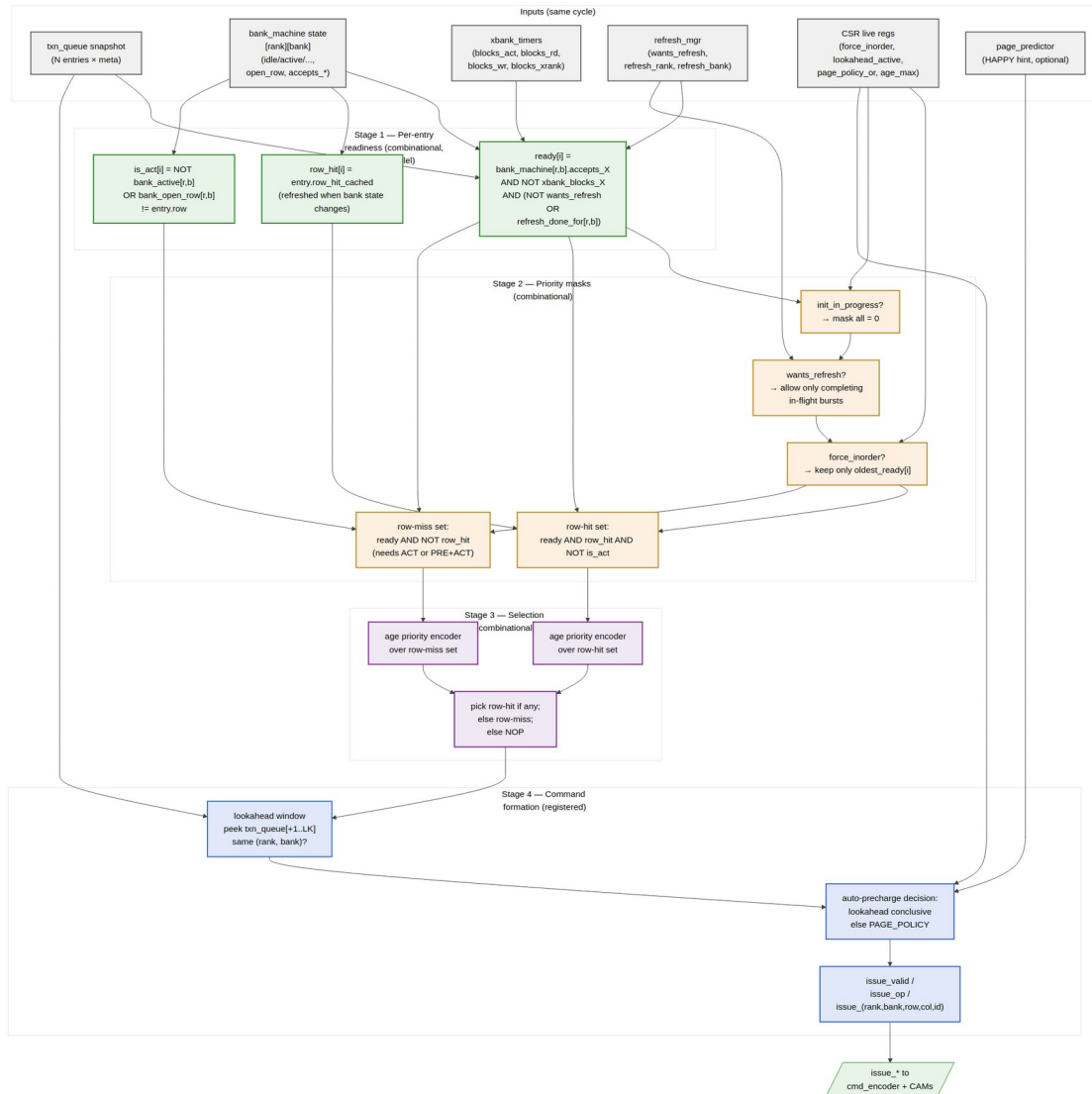
### 12.3.2 Outputs

| Signal        | Direction | Width                      | Description                                                                 |
|---------------|-----------|----------------------------|-----------------------------------------------------------------------------|
| issue_valid_o | output    | 1                          | A command is being issued this cycle                                        |
| issue_op_o    | output    | 4                          | One of {NOP, ACT, RD, RDA, WR, WRA, PRE, PREA, REF, REFPB, MRS, ZQCS, ZQCL} |
| issue_rank_o  | output    | $\text{clog}_2(\text{NR})$ | Target rank                                                                 |
| issue_bank_o  | output    | $\text{clog}_2(\text{NB})$ | Target bank                                                                 |
| issue_row_o   | output    | ROW_WIDTH                  | Target row (for ACT) or open-row pass-through (for RD/WR)                   |
| issue_col_o   | output    | COL_WIDTH                  | Target column (for RD/WR)                                                   |

| Signal                 | Direction | Width                             | Description                                                        |
|------------------------|-----------|-----------------------------------|--------------------------------------------------------------------|
| issue_axi_id_o         | output    | AXI_ID_WIDTH                      | Originating AXI ID (passed to the CAM for completion routing)      |
| issue_cam_slot_o       | output    | $\text{clog2}(\text{CAM\_DEPTH})$ | Index into rd_cmd_cam or wr_cmd_cam to mark “issued”               |
| issue_is_write_o       | output    | 1                                 | Picks wr_cmd_cam vs rd_cmd_cam for the mark-issued strobe          |
| predict_query_o        | output    | struct                            | (rank, bank, row) query to page_predictor_fub for stage-4 auto-pre |
| dbg_inorder_active_o   | output    | 1                                 | (debug) High when current cycle picked the in-order path           |
| dbg_refresh_blocking_o | output    | 1                                 | (debug) High when wants_refresh reduced the candidate pool         |

## 12.4 Decision Pipeline

The decision is a single MC-clock-cycle combinational pipeline broken into four conceptual stages, finishing in a registered output. Stages 1–3 are combinational; Stage 4 latches the final command into output flops.



## Scheduler Priority Pipeline

Source: [05\\_scheduler\\_priority\\_pipeline.mmd](#)

### 12.4.1 Stage 1 — Per-Entry Readiness (combinational, N parallel comparators)

For each of `TXN_QUEUE_DEPTH` entries, in parallel:

```
ready[i] = q_entries[i].valid
          AND (q_entries[i].state == PENDING)
          AND bank_machine[r,b].accepts_X
          AND NOT xbank_blocks_X
          AND ( NOT refresh_wants
                OR bank_machine[r,b].in_refresh_done_set )
```



```
is_row_hit[i] = q_entries[i].row_hit_cached
                ( cached at queue insert; refreshed when
bank_machine[r,b]
                state changes – see "Cache Coherency" below )
```

```
needs_act[i] = NOT bank_active[r,b]
                OR  bank_open_row[r,b] != q_entries[i].row
```

Where  $X$  is  $rd$  if  $is\_write == 0$ ,  $wr$  if  $is\_write == 1$ .  $r = q\_entries[i].rank$ ,  $b = q\_entries[i].bank$ .

$ready[i]$  answers “could I issue *some* command for this entry this cycle without violating any per-bank or cross-bank timer?”  $is\_row\_hit[i]$  answers “if I do, is it a column command on the already-open row?”  $needs\_act[i]$  answers “is the preliminary command actually ACT (or PRE+ACT) rather than RD/WR?”

### 12.4.2 Stage 2 — Priority Masking (combinational)

Five masks, computed in parallel from the Stage-1 vector and the config inputs:

| Mask          | Definition                                                                         | Effect                                                                                                                                                       |
|---------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| init_mask     | $init\_in\_progress ? 0 : ready[i]$                                                | Block all normal entries during init                                                                                                                         |
| refresh_mask  | $refresh\_wants ? (ready[i] \text{ AND } bank\_in\_refresh\_done\_set) : ready[i]$ | Allow only entries whose target rank+bank has already been granted the refresh handshake (i.e., already transitioned out of REFRESHING, or never entered it) |
| inorder_mask  | $force\_inorder ? oldest\_ready\_one\_hot : ready[i]$                              | Collapse to first-ready FIFO                                                                                                                                 |
| row_hit_mask  | $ready[i] \text{ AND } is\_row\_hit[i] \text{ AND } NOT\ needs\_act[i]$            | Row-hit candidates (column-only)                                                                                                                             |
| row_miss_mask | $ready[i] \text{ AND } (NOT\ is\_row\_hit[i] \text{ OR } needs\_act[i])$           | Row-miss candidates (need ACT or PRE+ACT)                                                                                                                    |

The composed candidate vectors are:

```
cands_row_hit = init_mask AND refresh_mask AND inorder_mask AND
row_hit_mask
cands_row_miss = init_mask AND refresh_mask AND inorder_mask AND
row_miss_mask
```

### 12.4.3 Stage 3 — Selection (combinational priority encoders)

Two parallel age-priority encoders, then a 2:1 mux:

```
winner_hit  = age_pe(cands_row_hit)    // index of oldest entry in
cands_row_hit, or none
winner_miss = age_pe(cands_row_miss)  // index of oldest entry in
cands_row_miss, or none

picked = winner_hit  IS_VALID ? winner_hit
        : winner_miss IS_VALID ? winner_miss
        : NONE      // issue NOP
```

The age priority encoder is the slowest path in this stage — it's an N-entry tournament on  $\text{clog2}(\text{AGE\_MAX}) = 8$ -bit ages. For  $N = 16$  (the default), that's 4 levels of pairwise compare-and-select. Synthesizable in two LUT layers per level on 7-series; ~30 LUT-delays in the typical config.

### 12.4.4 Stage 4 — Command Formation (registered)

Once an entry is picked, Stage 4 turns it into a DRAM command. This stage **may pre-flop intermediate signals** for timing closure on the highest clock targets (see “Pipeline Staging” below).

1. **Lookahead peek** — examine the next `cfg_lookahead_active` queue entries *after* the picked one to see if any target the same (rank, bank).
2. **Auto-precharge decision** — per HAS §3.2 (lookahead first, fallback to `PAGE_POLICY`):
  - Same-bank entry found AND its row matches the open row → **keep open** → bare RD / WR
  - Same-bank entry found AND its row differs → **close** → RDA / WRA
  - No same-bank entry within window → consult `cfg_page_policy_or`:
    - OPEN → bare RD / WR
    - CLOSE → RDA / WRA
    - HAPPY\_HYBRID → query `predict_hit_i[r][b]`; if predicted hit → bare; else RDA/WRA
3. **Op encoding** — generate the 4-bit `issue_op_o` from (`is_write`, `needs_act`, `auto_pre`):
  - `needs_act` and bank state == IDLE → ACT
  - `needs_act` and bank state == ACTIVE (different row) → PRE (the column command follows next cycle after tRP)
  - NOT `needs_act` and `is_write` → WR or WRA
  - NOT `needs_act` and NOT `is_write` → RD or RDA

#### 4. **Latch** outputs into the issue flops.

### 12.4.5 Refresh Window Behavior

When `refresh_wants_i` is high, `refresh_mgr_fub` is asserting `refresh_req` to one or more bank machines. Those bank machines drive `refresh_done_for_i[r][b] = 1` only after reaching IDLE and handing over the grant. While `refresh_done_for_i[r][b] == 0`, that bank's entries are masked out — no new column commands to a bank that's about to refresh.

Bank machines for *other* ranks (in the multi-rank REFab round-robin) or *other* banks (in REFpb) remain available, so the scheduler continues to drain those during the refresh window. This is what makes per-rank REFab dispatch beneficial — the non-refreshing ranks keep their throughput.

When all required refreshes complete and `refresh_wants_i` drops, the masks lift and normal FR-FCFS resumes the next cycle.

---

## 12.5 Cache Coherency: `row_hit_cached`

---

`q_entries[i].row_hit_cached` is computed at queue insertion (by `axi4_slave_fub` via a one-cycle query to `bank_machine[r,b].open_row`) and stored in the queue entry. This avoids re-querying every cycle. But the bank state can change between insertion and selection:

- When `bank_machine[r,b]` transitions ACTIVE → ACTIVE on a different row (via PRE + ACT to a new row), all `q_entries` with `(rank, bank) == (r, b)` need their `row_hit_cached` recomputed against the new `open_row`.
- When `bank_machine[r,b]` transitions IDLE → ACTIVATING, the queue entries get `row_hit_cached = 1` for entries whose row matches the in-flight ACT.

This is implemented as a **broadcast update**: on every bank state transition that affects the open row, the bank machine emits `open_row_changed[r][b]` + the new `open_row`. The queue has a per-entry update path that, in parallel, recomputes `row_hit_cached`. This is a one-cycle update — the recomputed value is visible to the scheduler in the cycle *after* the bank transition.

The one-cycle lag means the scheduler may make an “outdated row-hit” decision once per row change. The downside is one mis-categorized RD/WR (issued as bare when RDA was warranted, or vice versa); the worst case is one extra PRE later. This is a deliberate trade — the alternative is a combinational broadcast from bank state through every queue entry's row-hit comparator, which would inflate the Stage-1 critical path.

---

## 12.6 Pipeline Staging

---

The default build is a **single-cycle combinational** scheduler — Stages 1–3 are combinational; Stage 4's auto-precharge decoder is combinational; only the final issue-flop is registered. On 7-series FPGA at 200 MHz this closes timing comfortably. For 14 nm targets at 500+ MHz, two timing-closure escape hatches:

1. **Stage 3 → Stage 4 flop insertion** — register picked between stages. Adds one cycle of issue latency. Throughput is unchanged because the pipeline is single-issue anyway; the only visible difference is that a request inserted at cycle T can first issue at cycle T+3 instead of T+2.
2. **Stage 1 partition by bank** — partition the readiness comparator network by (rank, bank) so each partition produces a per-bank ready vector independently, then aggregate. Useful when  $\text{NUM\_RANKS} \times \text{NUM\_BANKS}$  is small (the default  $1 \times 8 = 8$  partitions) and routing congestion dominates.

These are not on by default; they're synthesis-script switches, not RTL parameters.

---

## 12.7 Strict In-Order Mode (collapse)

---

When `cfg_force_inorder_i = 1` (and `SCHEDULER_MODE == "000"` is synthesized):

- `inorder_mask` collapses to `oldest_ready_one_hot` — a one-hot vector that picks the *oldest valid ready entry* by age.
- This drops the row-hit prioritization (no row-hit vs row-miss split — both go through `cands_row_miss` since age order rules).
- Refresh priority is preserved (JEDEC requirement).
- Lookahead and HAPPY remain on at elaboration but can be independently disabled at runtime (`cfg_lookahead_active = 0`, `cfg_happy_enable = 0`) for a pure first-ready FIFO.

When `SCHEDULER_MODE == "INORDER"` is synthesized:

- The Stage-2 / Stage-3 priority encoders are different RTL: the row-hit/row-miss split is omitted; only the oldest-ready selector is built.
  - `cfg_force_inorder_i` becomes a tied observation bit at `STATUS.force_inorder_obs = 1`.
-

## 12.8 Critical-Path Analysis (rough budget at 500 MHz target, 2 ns cycle)

| Path                                                                    | Levels (approx)           | Budget        |
|-------------------------------------------------------------------------|---------------------------|---------------|
| bank_state_i.accepts_X → ready[i] → row-hit mask → cands_row_hit[i]     | 4 LUT levels              | 0.7 ns        |
| cands_row_hit → age priority encoder → winner_hit (16-entry tournament) | 4 levels × 2 LUT each = 8 | 1.2 ns        |
| winner_hit / winner_miss → 2:1 mux → picked                             | 1 LUT level               | 0.2 ns        |
| Routing / setup margin                                                  |                           | 0.3 ns        |
| <b>Total Stage 1–3</b>                                                  |                           | <b>2.4 ns</b> |

So at 500 MHz the path **misses by ~400 ps** and needs Stage 3 → Stage 4 flop insertion (the first escape hatch above). At 200 MHz target (5 ns cycle, embedded SoC default), the path closes with ~2.6 ns of slack — no escape hatch needed.

The 500 MHz analysis is FPGA-pessimistic; ASIC targets close 500 MHz on this path comfortably with standard pipelining.

## 12.9 CSR Hooks

The scheduler drives several CSR observability fields:

| CSR field                       | Source signal                                          | Use case                                           |
|---------------------------------|--------------------------------------------------------|----------------------------------------------------|
| STATUS.issue_op_obs (R)         | Last issued issue_op_o                                 | Bring-up debug: what was the last DRAM command     |
| STATUS.scheduler_idle_pct (R)   | Rolling counter — fraction of cycles issue_valid_o = 0 | Bandwidth-headroom telemetry                       |
| OBS_ROW_HIT_RANK<R>_BANK<N> (R) | Incremented when row-hit path wins for that bank       | Per-bank row-hit rate (HAS §6.3)                   |
| OBS_INORDER_FALLBACKS (R)       | Incremented when force_inorder ate a row-hit           | Tells software how much OoO is currently giving up |

| CSR field                        | Source signal                                        | Use case                                               |
|----------------------------------|------------------------------------------------------|--------------------------------------------------------|
|                                  | win                                                  |                                                        |
| OBS_LOOKAHEAD_HI<br>TS (R)       | Incremented when Stage-4<br>lookahead was conclusive | Tunes<br>LOOKAHEAD_DEPTH_MAX<br>characterization sweep |
| STATUS.force_ino<br>rder_obs (R) | Echo of effective force-<br>inorder state            | Software-readable<br>confirmation                      |

## 12.10 Verification Notes (cocotb test plan)

| Scenario                                                                      | What it proves                                               |
|-------------------------------------------------------------------------------|--------------------------------------------------------------|
| Single-write, single-bank, single-rank                                        | Smoke test: ACT → WR →<br>b_complete cycle                   |
| Single-read, single-bank, single-rank                                         | Smoke test: ACT → RD → beat<br>return                        |
| Two reads, same bank, same row, oldest<br>first                               | Row-hit reordering not invoked<br>(already in order)         |
| Two reads, same bank, <i>different</i> rows,<br>oldest is row-conflict        | OoO: younger row-hit wins over<br>older row-miss             |
| Same as above with force_inorder = 1                                          | OoO disabled: older row-miss wins<br>despite younger row-hit |
| Cross-bank: rd to bank 0 and rd to bank 1,<br>bank 0 row-miss, bank 1 row-hit | Bank parallelism: bank-1 issue<br>fires first                |
| Cross-rank (NUM_RANKS=2): rd to rank 0<br>bank 3 and rd to rank 1 bank 3      | Per-rank bank parallelism                                    |
| Refresh during in-flight burst                                                | Refresh window correctly drains<br>and resumes               |
| Refresh during multi-rank traffic (REFab<br>on rank 0, traffic on rank 1)     | Per-rank REFab dispatch: rank-1<br>traffic continues         |
| Age-saturation under sustained row-<br>conflict from younger IDs              | Older row-conflict eventually wins<br>via age priority       |
| LOOKAHEAD_DEPTH_MAX = 4, lookahead<br>concludes auto-precharge correctly      | Lookahead path                                               |
| LOOKAHEAD_DEPTH_MAX = 0, HAPPY                                                | Fallback path through page                                   |

| Scenario                                              | What it proves                        |
|-------------------------------------------------------|---------------------------------------|
| predictor used                                        | predictor                             |
| PAGE_POLICY = CLOSE, every column command is auto-pre | CLOSE policy                          |
| Init-in-progress blocks normal traffic                | init_mask gating                      |
| Queue full + back-pressure to AXI                     | axi4_slave does not accept new bursts |

## 12.11 Open Questions / Future Work

- Should QoS (awqos / arqos) be promoted from the v1 “no behavior” pass-through to a real priority boost in the FR-FCFS function? Currently HAS §3.2 leaves QoS as a side-band hint for v2. The hook would be a qos\_boost\_mask between the row-hit and row-miss stages.
- The cross-rank read-to-write turnaround (tRTRS + write window) lives in xbank\_timers\_fub today; the scheduler consumes the gate but does not pre-plan around it. A smarter scheduler could **batch by rank** (issue all ready reads on rank 0, then all on rank 1) to amortize tRTRS. This is a v2 feature; v1 takes the simpler per-cycle gating.
- Whether OBS\_INORDER\_FALLBACKS is useful for characterization is not yet validated; the counter is cheap, but if it’s never read by the bring-up team it can be dropped in v2.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

## 13 HAPPY Page Predictor (page\_predictor\_fub)

**Module:** page\_predictor\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Synthesized only when PAGE\_POLICY == HAPPY\_HYBRID. See HAS §3.2 page\_predictor for algorithm details (Ghasempour et al. 2015).

## 13.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 14 Bank Machine (bank\_machine\_fub)

**Module:** bank\_machine\_fub.sv **Location:** rtl/fub/ **Category:** FUB **Parent:** ddr2\_lpddr2\_ctrl (instantiated NUM\_RANKS × NUM\_BANKS times) **Status:** Draft v0.1

Architectural context: HAS §3.3. The FSM state diagram is in ddr2\_lpddr2\_has/assets/mermaid/03\_bank\_machine\_fsm.png and is the canonical state reference — this section is the implementation view (port interface, counter pool, broadcast updates, multi-rank instantiation, timing).

---

### 14.1 Purpose

---

bank\_machine\_fub is the per-(rank, bank) state machine that enforces JEDEC per-bank timing constraints. One FSM instance per (rank, bank) — so a 1-rank 8-bank build has 8 instances, a 2-rank 8-bank build has 16, a 4-rank 8-bank build has 32. Each instance tracks:

- Open-row state and the row identifier
- All per-bank timing counters (tRCD, tRAS, tRP, tRC, tWR, tRFCpb, plus per-RD CL and per-WR CWL windows)
- The “last refreshed” age used by DARP per-bank refresh selection
- The refresh handshake state to refresh\_mgr\_fub



The instance fans its outputs into:

- A per-(rank, bank) row of the scheduler's bank-state matrix
- The txn\_queue's row-hit-cache refresh broadcast (`open_row_changed[r][b]` + `new_open_row[r][b]`)
- The refresh manager's `refresh_gnt[r][b]` array and the `last_ref_age[r][b]` DARP selection input

The bank machine is the most-replicated FUB in the design. Per-instance area is small (~150–300 LUTs, see below), but the replication factor ( $NR \times NB$ , up to 32) makes total bank-machine area the second-largest after the scheduler.

---

## 14.2 Synthesis Parameters

| Parameter          | Source  | Effect on this FUB                                                                                                               |
|--------------------|---------|----------------------------------------------------------------------------------------------------------------------------------|
| ROW_WIDTH          | top     | Width of <code>open_row</code> register and <code>new_open_row</code> broadcast                                                  |
| COL_WIDTH          | top     | Width of column passthrough (consumed by <code>cmd_encoder</code> , not stored)                                                  |
| MEMTYPE            | top     | Controls whether the REFRESHING state uses <code>tRFCab</code> or <code>tRFCpb</code> (LPDDR2 only has <code>REFpb</code> )      |
| T*_WIDTH_MAX       | derived | Width of each timing-counter register (set by max CSR-loaded <code>tREFI</code> , <code>tRC</code> , <code>tRFCpb</code> , etc.) |
| LAST_REF_AGE_WIDTH | derived | Width of <code>last_ref_age</code> counter; sized to span $8 \times tREFI\_cycles$ without saturation                            |

The bank machine takes **no rank/bank identifier as a parameter** — every instance is identical. Identity comes from the position in the top-level generate block and is passed in via wire-level `bank_id_i` and `rank_id_i` ports (used only for assertion messages and CSR routing; the FSM logic is identity-agnostic).

---

## 14.3 Instantiation Pattern

Recap from §2.1 top-integration:

```
generate
  for (genvar r = 0; r < NUM_RANKS; r++) begin : g_rank
    for (genvar b = 0; b < NUM_BANKS; b++) begin : g_bank
```

```

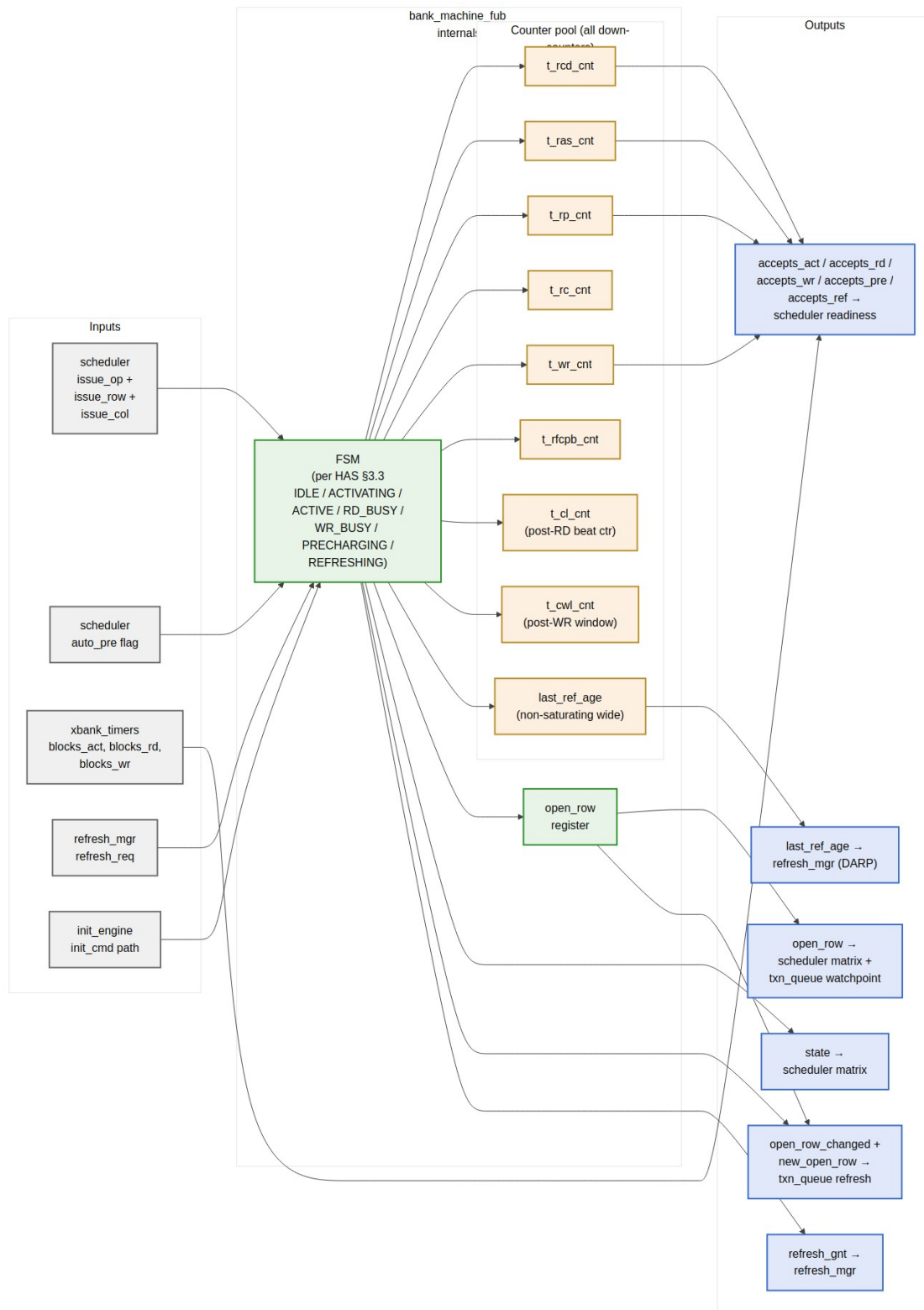
bank_machine_fub #(.ROW_WIDTH(ROW_WIDTH)) u_bank_machine (
    .mc_clk          (mc_clk),
    .mc_rst_n        (mc_rst_n),
    .rank_id_i       (r[$clog2(NR)-1:0]),
    .bank_id_i       (b[$clog2(NB)-1:0]),
    .sched_issue_if  (sched_to_bank[r][b]),
    .xbank_blocks_i  (xbank_to_bank[r][b]),
    .refresh_req_i   (refresh_req[r][b]),
    .refresh_gnt_o   (refresh_gnt[r][b]),
    .init_cmd_if     (init_to_bank[r][b]),
    .state_o         (bank_state[r][b]),
    .open_row_o      (bank_open_row[r][b]),
    .accepts_o       (bank_accepts[r][b]),
    .open_row_changed_o (orw_changed[r][b]),
    .new_open_row_o  (new_orw[r][b]),
    .last_ref_age_o  (bank_last_ref_age[r][b])
);
end
endgenerate

```

The outputs are 2-D-indexed arrays at the top level. The scheduler and txn\_queue consume them by name; no aggregation logic in the top — pure structural wiring.

---

## 14.4 Block Internals



Source: [07\\_bank\\_machine\\_internals.mmd](#)

The diagram above is the implementation view. The HAS state-diagram view is at `ddr2_lpddr2_has/assets/mermaid/03_bank_machine_fsm.png` — refer to it for state transitions; this section is the port + counter detail.

---

## 14.5 FSM States and Transition Triggers

---

Recap of the seven states (full state diagram in HAS):

| State       | Held while                                                   | Exits via                                                              |
|-------------|--------------------------------------------------------------|------------------------------------------------------------------------|
| IDLE        | No row open; bank free                                       | ACT or REFpb issued by scheduler                                       |
| ACTIVATING  | $t\_rcd\_cnt > 0$                                            | $t\_rcd\_cnt == 0 \rightarrow ACTIVE$                                  |
| ACTIVE      | Row open; awaiting column command or PRE                     | RD/RDA, WR/WRA, or PRE issued by scheduler                             |
| RD_BUSY     | $t\_cl\_cnt > 0$ (CL + BL/2 beats remaining)                 | Last beat returned $\rightarrow ACTIVE$ (bare RD) or PRECHARGING (RDA) |
| WR_BUSY     | $t\_cwl\_cnt > 0$ (CWL + BL/2 + tWR window)                  | Window expired $\rightarrow ACTIVE$ (bare WR) or PRECHARGING (WRA)     |
| PRECHARGING | $t\_rp\_cnt > 0$                                             | $t\_rp\_cnt == 0 \rightarrow IDLE$                                     |
| REFRESHING  | $t\_rfc\_cnt > 0$ (tRFCpb for LPDDR2, tRFCab for DDR2 REFab) | $t\_rfc\_cnt == 0 \rightarrow IDLE$                                    |

**Auto-precharge handling.** RDA / WRA do not require a separate state — they are encoded as an “auto-pre pending” flag set when the FSM enters RD\_BUSY / WR\_BUSY. When the busy window expires, the flag steers the next-state to PRECHARGING instead of ACTIVE. No PRE command is consumed from the scheduler — the precharge timing is purely internal. This is the same trick the `cmd_encoder` uses to fold RDA into one DRAM command.

**Init-engine bypass.** During init, the `init_cmd_if` path can issue ACT, PRE, MRS, REF directly without going through the scheduler. The FSM honors these as if they came from the scheduler — same state transitions, same counter reloads. The init engine knows the JEDEC sequence; the bank machine just executes it.

## 14.6 Counter Pool

All counters are **down-counters that saturate at 0**. Each is loaded with a CSR-supplied cycle count on the relevant trigger.

| Counter      | Load Trigger                                     | Reload Value (cycles)               | Saturate<br>s At                              | Width   |
|--------------|--------------------------------------------------|-------------------------------------|-----------------------------------------------|---------|
| t_rcd_cnt    | ACT issued                                       | TIMINGS_RC_RCD_RP.tRCD              | 0                                             | 8 bits  |
| t_ras_cnt    | ACT issued                                       | TIMINGS_RC_RCD_RP.tRAS              | 0                                             | 8 bits  |
| t_rp_cnt     | PRE or PREA issued; or RDA/WRA window expiration | TIMINGS_RC_RCD_RP.tRP               | 0                                             | 8 bits  |
| t_rc_cnt     | ACT issued                                       | TIMINGS_RC_RCD_RP.tRC               | 0                                             | 8 bits  |
| t_wr_cnt     | WR or WRA last beat issued                       | TIMINGS_CL_CWL_WR.tWR               | 0                                             | 8 bits  |
| t_rfcpb_cnt  | REF or REFpb issued                              | LPDDR2 tRFCpb / DDR2 tRFCab         | 0                                             | 16 bits |
| t_cl_cnt     | RD or RDA issued                                 | TIMINGS_CL_CWL_WR.CL + BL/2         | 0                                             | 8 bits  |
| t_cwl_cnt    | WR or WRA issued                                 | TIMINGS_CL_CWL_WR.CWL + BL/2 + tWR  | 0                                             | 8 bits  |
| last_ref_age | REF or REFpb completion (t_rfcpb_cnt → 0)        | reload to 0; otherwise +1 per cycle | does not saturate (wide enough for 8 × tREFI) | 24 bits |

The CSR-supplied values are read from TIMINGS\_\* registers in csr\_apb\_fub; the values are live (not staged) since timing parameters are *typically* loaded once at init and not changed at runtime. Software *can* change them at quiet points, but the bank machine doesn't enforce quiet-point staging — it just samples them on each counter load.

**last\_ref\_age** is **non-saturating** because DARP needs to distinguish an entry that was refreshed 10 cycles ago from one refreshed 10,000 cycles ago. A 24-bit width supports  $2^{24} / 200 \text{ MHz} = 84 \text{ ms}$  of age, well past  $8 \times t_{\text{REFI}}$  (typically 50  $\mu\text{s}$ ).

---

## 14.7 Outputs to Scheduler (accepts\_\*)

---

The `accepts_o` struct is the **combinational readiness output** consumed by the scheduler's Stage-1 readiness computation. Each member is the AND of "FSM state permits this command" and "all relevant counters are 0" and "xbank constraints permit this command":

```
accepts_act = (state == IDLE)
               AND (t_rp_cnt == 0)
               AND (t_rc_cnt == 0)
               AND NOT xbank_blocks_act
```

```
accepts_rd  = (state == ACTIVE)
               AND (t_rcd_cnt == 0)
               AND NOT xbank_blocks_rd
```

```
accepts_wr  = (state == ACTIVE)
               AND (t_rcd_cnt == 0)
               AND NOT xbank_blocks_wr
```

```
accepts_pre = (state == ACTIVE)
               AND (t_ras_cnt == 0)
```

```
accepts_ref = (state == IDLE)
               AND (t_rp_cnt == 0)
               AND (t_rc_cnt == 0)
```

These are pure combinational — no flop between the state register and the scheduler. The scheduler's Stage-1 critical path (per §2.7) routes through these comparators.

**Why xbank lives outside.** Cross-bank constraints ( $t_{\text{RRD}}$ ,  $t_{\text{FAW}}$ ,  $t_{\text{CCD}}$ ,  $t_{\text{WTR}}$ ,  $t_{\text{RTW}}$ ) are shared across all banks of a rank or globally across the channel — they can't be enforced inside a per-bank FSM. The `xbank_timers_fub` (§2.10) aggregates them and presents the per-(rank, bank) AND-mask via `xbank_blocks_i`.

---

## 14.8 Refresh Handshake Protocol

---

The bank machine cooperates with `refresh_mgr_fub` via a two-wire handshake per (rank, bank):

| Signal        | Direction          | Meaning                                                              |
|---------------|--------------------|----------------------------------------------------------------------|
| refresh_req_i | refresh_mgr → bank | “Drain to a quiet state and await my REF / REFpb command”            |
| refresh_gnt_o | bank → refresh_mgr | “I am in IDLE (or REFRESHING) and will not issue any column command” |

The bank machine’s protocol:

1. **refresh\_req asserts** while the bank is in ACTIVE / RD\_BUSY / WR\_BUSY: the bank does not assert refresh\_gnt yet. It finishes its current column command (if any) and falls back to ACTIVE. The scheduler is masked from issuing new column commands to this bank during the refresh window (txn\_queue watchpoint — see §2.6 and §2.7).
2. **refresh\_req asserts** while the bank is ACTIVE and no column command is in flight: the bank issues an internal precharge (driving accepts\_pre high to signal the scheduler can issue PRE; or, if the scheduler is gated for the refresh window, the bank can self-issue PRE on t\_ras\_cnt == 0). Once state == IDLE and t\_rp\_cnt == 0, refresh\_gnt asserts.
3. **refresh\_req asserts** while the bank is in IDLE: refresh\_gnt asserts the same cycle.
4. **REF/REFpb arrives** from the scheduler (via the refresh manager’s priority path): FSM transitions IDLE → REFRESHING, t\_rfcpb\_cnt loads, last\_ref\_age resets to 0.
5. **refresh\_req deasserts** after refresh completes: bank machine drops refresh\_gnt, transitions REFRESHING → IDLE on t\_rfcpb\_cnt == 0, and is free to accept new ACTs.

For multi-rank REFab dispatch (per §2.11 refresh\_mgr), the refresh manager asserts refresh\_req[r][\*] for all banks on the target rank simultaneously and waits for all refresh\_gnt[r][\*] before issuing REF. Other ranks’ bank machines see refresh\_req[r' ≠ r] = 0 throughout and continue normal operation.

---

## 14.9 Open-Row Change Broadcast

---

When the FSM transitions in a way that changes the open row, the bank machine asserts a one-cycle broadcast to txn\_queue\_fub:

| Transition                                                  | open_row_changed_o                                    | new_open_row_o  |
|-------------------------------------------------------------|-------------------------------------------------------|-----------------|
| IDLE → ACTIVATING (via ACT)                                 | 1 cycle                                               | The ACT's row   |
| ACTIVE → PRECHARGING → IDLE (after RDA/WRA or explicit PRE) | 1 cycle (on entering IDLE)                            | 0 (no row open) |
| REFRESHING → IDLE                                           | 1 cycle                                               | 0 (no row open) |
| ACTIVATING → ACTIVE                                         | not asserted (row was already valid as of ACTIVATING) | —               |

This is the path that drives `txn_queue`'s `row_hit_cached` refresh, as described in §2.6. The one-cycle lag (broadcast at T, queue refresh visible at T+1) is the deliberate trade described in both this block and the scheduler (§2.7).

`new_open_row_o = 0` when the bank is leaving an open-row state is a convention — the queue's refresh path compares against the new row only when the bank ends in an active state. When ending in IDLE, the queue sets `row_hit_cached = 0` for all entries targeting this (rank, bank) regardless of `new_open_row_o`.

---

## 14.10 Per-Instance vs. Aggregated Storage

---

What lives per-instance ( $NR \times NB$  copies):

- state register (3 bits)
- open\_row register (ROW\_WIDTHH bits)
- All counters (~80 bits per instance)
- auto\_pre\_pending flag (1 bit)

What is aggregated at the top level (one copy each):

- `bank_state[NR][NB]` — concatenation of per-instance state outputs into a 2-D array
- `bank_accepts[NR][NB]` — concatenation of accepts structs
- The DARP candidate-set vector (computed by `refresh_mgr_fub` from the array of `last_ref_age` outputs)
- The scheduler's match vector (computed by the scheduler from `bank_state` and `bank_accepts`)



This separation matters for synthesis: the per-instance flops are local to each bank machine (good for placement); the aggregation buses are wide ( $NR \times NB \times \sim 30$  bits = up to  $\sim 1000$  bits at  $4 \times 8$ ) and route across the chip to the scheduler and refresh manager.

**Routing concern.** At  $NUM\_RANKS=4$ ,  $NUM\_BANKS=8$ , the bank-state array routes 32 instances' worth of state and accepts wires to the scheduler. This is a  $\sim 1000$ -bit bus and is the second-widest in the design (after `txn_queue.q_entries_o`). The scheduler's Stage-1 critical path absorbs this in the 200 MHz target with slack; the 500 MHz target requires registered bank-state outputs (which adds one cycle to the scheduler-to-issue latency but is otherwise harmless — bank states change slowly compared to scheduler issue).

## 14.11 Per-Instance Area Estimate

At default config ( $ROW\_WIDTH=14$ ,  $COL\_WIDTH=10$ , 8-bit timing counters, 24-bit `last_ref_age`):

| Item                                                                      | Per instance                                                                   |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| FSM state (3 bits + 3-bit next)                                           | $\sim 10$ LUTs                                                                 |
| Counter pool (8 down-counters)                                            | $\sim 60$ flops + 30 LUTs                                                      |
| <code>open_row</code> register (14 bits)                                  | 14 flops                                                                       |
| <code>last_ref_age</code> (24 bits)                                       | 24 flops                                                                       |
| <code>auto_pre_pending</code> + combinational <code>accepts_* glue</code> | $\sim 20$ LUTs                                                                 |
| <b>Per-instance total</b>                                                 | <b><math>\sim 120</math> flops + 60 LUTs <math>\approx</math> 100–150 LUTs</b> |
| 32-instance total ( $4\text{-rank} \times 8\text{-bank}$ )                | <b><math>\sim 3,200\text{--}4,800</math> LUTs</b>                              |

This is  $\sim 25\%$  of the controller's total LUT budget at the max-rank config — significant. At single-rank, the bank machines fall to  $\sim 800\text{--}1,200$  LUTs, comfortably in budget.

## 14.12 CSR Hooks

| CSR field                                                    | Source signal                                                  | Use case                              |
|--------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------|
| <code>OBS_ROW_HIT_RANK&lt;R&gt;_BANK&lt;N&gt; (R)</code>     | Driven by scheduler when row-hit picks this bank               | Per-bank row-hit telemetry (HAS §6.3) |
| <code>OBS_REF_LATENCY_RANK&lt;R&gt;_BANK&lt;N&gt; (R)</code> | Time from <code>refresh_req</code> to <code>refresh_gnt</code> | Per-bank refresh-blocking time        |
| <code>OBS_BANK_OPEN_ROW_R&lt;R&gt;_B&lt;N&gt; (R)</code>     | Current <code>open_row</code> value                            | Debug: which row is currently open    |

| CSR field                    | Source signal                           | Use case                                |
|------------------------------|-----------------------------------------|-----------------------------------------|
| OBS_BANK_STATE_R<R>_B<N> (R) | Current FSM state (3-bit encoding)      | Debug: bank state for bring-up          |
| STATUS.bank_idle_mask (R)    | One bit per (rank, bank): state == IDLE | Quick visibility into bank availability |

The per-(rank, bank) observation registers are sparse at multi-rank — at NR=4, NB=8 there are 32 of each. The CSR slave generates them from a YAML template (see §4.2). Software queries the cap\_max\_ranks and NUM\_BANKS capability bits (§4.4) to know which registers exist.

## 14.13 Verification Notes (cocotb test plan)

| Scenario                                                                        | What it proves                                            |
|---------------------------------------------------------------------------------|-----------------------------------------------------------|
| Single ACT → RD → PRE → IDLE on bank 0                                          | Smoke: each counter loads and decrements correctly        |
| ACT → RDA: bank auto-precharges without explicit PRE from scheduler             | Auto-precharge flag steers RD_BUSY → PRECHARGING          |
| ACT → WRA: bank auto-precharges; t_wr_cnt window respected                      | Write recovery before precharge                           |
| Scheduler issues RD while t_rcd_cnt > 0                                         | Assertion fires; accepts_rd was 0 — should be impossible  |
| refresh_req asserts during ACTIVATING                                           | refresh_gnt waits until bank reaches IDLE                 |
| refresh_req asserts during RD_BUSY                                              | Bank completes RD, returns to ACTIVE, then drains via PRE |
| refresh_req asserts when bank is IDLE                                           | refresh_gnt asserts same cycle                            |
| REF completion → last_ref_age resets to 0                                       | DARP age-tracking correctness                             |
| Two adjacent ACTs to different rows on same bank — second blocks on t_rp_cnt    | Per-bank tRP enforcement                                  |
| Cross-bank ACT (handled by xbank_timers) — should NOT affect this bank          | Per-bank FSM is rank/bank-local                           |
| Multi-rank (NR=2): refresh on rank 0 bank 3; rank 1 bank 3 continues normal ops | Per-rank refresh isolation                                |
| Open-row change broadcast: ACT issued, open_row_changed asserts for 1 cycle     | Broadcast timing correctness                              |

| Scenario                                                          | What it proves |
|-------------------------------------------------------------------|----------------|
| Soft reset during WR_BUSY — counters clear, state returns to IDLE | Reset behavior |

## 14.14 Open Questions / Future Work

- **CSR coverage of per-bank state.** At NR=4, NB=8 the OBS\_BANK\_OPEN\_ROW\_\* registers consume 32 CSR slots — non-trivial. Worth gating behind a cap\_per\_bank\_obs capability bit so smaller builds skip them entirely?
- **Per-instance vs centralized last\_ref\_age.** Could the age counters live in refresh\_mgr\_fub and free up flops per bank machine? Slight area win but adds a wide read port from refresh\_mgr to bank\_machine for age comparison; punt to v2.
- **Auto-pre flag clear timing.** Currently the auto\_pre\_pending flag clears on entering PRECHARGING. If a refresh request arrives mid-RD with auto-pre pending, the bank still does the auto-pre (drains via tRP) before granting refresh. This is correct per JEDEC, but worth a verification scenario (added above).
- **tWR vs tRP composition.** A WRA → PRECHARGING transition needs  $\max(t_{wr\_cnt}, t_{rp\_load})$  cycles. Implementation: the FSM enters PRECHARGING after  $t_{wr\_cnt}$  expires and *then* loads  $t_{rp\_cnt}$ . This is the simplest correct sequence; alternative is to start  $t_{rp\_cnt}$  early and gate IDLE on  $\max(t_{wr\_cnt}, t_{rp\_cnt})$ . The simpler form is preferred unless characterization shows it costs measurable bandwidth.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

## 15 Cross-Bank Timers (xbank\_timers\_fub)

**Module:** xbank\_timers\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-rank tRRD/tFAW, global tCCD/tWTR/tRTW, cross-rank tRTRS/tCS. See HAS §3.3 xbank\_timers and the Per-Rank vs. Cross-Rank Scope table.

## 15.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 16 Refresh Manager (refresh\_mgr\_fub)

**Module:** refresh\_mgr\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Round-robin rank dispatch, DARP bank selection, postponer, periodic ZQCS, per-rank PASR mask. See HAS §3.4.

### 16.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 17 Init Engine (`init_engine_fub`)

**Module:** `init_engine_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

Microprogram-driven cold-boot. Step-table ROM in `rtl/macro/` per memtype. See HAS §3.5.

### 17.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 18 Power-State FSM (`power_state_fub`)

**Module:** `power_state_fub.sv` **Location:** `rtl/fub/` **Status:** Skeleton — to be authored during MAS week

Active / Active-PD / Self-Refresh / Deep-PD; per-rank CKE; SR coordination; quiet-point detector. See HAS §3.5.

### 18.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in `assets/mermaid/`)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)

- Verification notes (cocotb test plan)

## 19 Command Encoder (cmd\_encoder\_fub)

**Module:** cmd\_encoder\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

DDR2 ras/cas/we vs LPDDR2 20-bit CA bus, selected at elaboration by MEMTYPE. See HAS §3.6.

### 19.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

## 20 DFI Gear (gear\_dfi\_fub)

**Module:** gear\_dfi\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-phase packing per N\_PHASES / WRPHASE / RDPHASE. See HAS §3.6 gear\_dfi.

## 20.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 21 ODT Control (odt\_ctrl\_fub)

**Module:** odt\_ctrl\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

Per-rank dfi\_odt[r] per ODT\_RULE\_MULTIRANK. The home of the cross-rank read/write ODT rules. See HAS §3.6.

### 21.1 TODO (Week-of-MAS work)

---

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 22 Write Data Path (wr\_data\_path\_fub)

**Module:** wr\_data\_path\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

AXI W → DFI wrdata. Width conversion AXI DW → NP × DFI\_DW. Strobe propagation. See HAS §3.7.

### 22.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)
- Verification notes (cocotb test plan)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 23 Read Data Path (rd\_data\_path\_fub)

**Module:** rd\_data\_path\_fub.sv **Location:** rtl/fub/ **Status:** Skeleton — to be authored during MAS week

DFI rddata → AXI R. CL/CWL-aware capture, ID-tagged OoO completion. See HAS §3.7.

### 23.1 TODO (Week-of-MAS work)

- Interface signal list (inputs/outputs, widths, parameter dependencies)
- Internal state and storage
- FSM diagram (mermaid in assets/mermaid/)
- Datapath timing and per-stage flops
- Timing budget on critical paths
- CSR hooks (observability registers driven by this FUB)



- Verification notes (cocotb test plan)

## 24 AXI4 Slave Protocol

**Status:** Skeleton — to be authored during MAS week

The AXI4 slave port carries host requests into the controller.

### 24.1 Scope of This Section

---

This section is the **wire-level** AXI4 contract specific to this controller — what's supported, what's relaxed, what's omitted. It is not an AXI4 tutorial; refer to ARM IHI 0022 for the full protocol.

### 24.2 Supported Features (v1)

---

- AW/W/B/AR/R 5-channel handshake
- INCR burst type (mandatory)
- ID-aware OoO completion (when `AXI_000_ACROSS_IDS = true`)
- 64 / 128 / 256-bit data widths
- 1–16-bit ID widths
- Per-ID in-order completion

### 24.3 Optional Features (build-time)

---

- FIXED burst (`SUPPORT_FIXED_BURST`)
- WRAP burst (`SUPPORT_WRAP_BURST`)

### 24.4 Unsupported Features

---

- AXI exclusive accesses
- AXI4 user signals (no `USER_*` widths)
- AXI4 cache-coherent extensions

### 24.5 TODO

---

- Per-channel timing diagrams (wavedrom)

- Backpressure semantics in detail
- Response-ordering examples for OoO-across-IDs
- Burst-boundary corner cases (4 KB boundary, address-alignment)

## 25 DFI v2.1 Master Protocol

**Status:** Skeleton — to be authored during MAS week

The DFI v2.1 master port drives the DRAM PHY.

### 25.1 Scope

---

This section is the wire-level DFI v2.1 contract as implemented by this controller. The full DFI v2.1 spec covers more — training, frequency change, low-power negotiation — see HAS §2.1 for what is and isn't in scope here.

### 25.2 Sub-Interfaces Supported

---

- Command sub-interface
- Write-data sub-interface
- Read-data sub-interface
- Status sub-interface
- Update sub-interface (controller-initiated and PHY-initiated)

### 25.3 Sub-Interfaces Not Supported (v1)

---

- Training (not required at v2.1 for DDR2/LPDDR2)
- Frequency change
- Low-power negotiation (CKE used directly)
- CRC (not in DDR2/LPDDR2)

### 25.4 Phase / Gear Topology

---

N\_PHASES per MC clock; gear FUB (see §2.15) handles packing.

## 25.5 TODO

---

- Per-signal timing diagrams (rise-to-rise)
- WRPULSE / RDPULSE selection guidelines
- ctrlupd / phyupd protocol detail (rare in v2.1 but spec-mandated)
- DDR2 ras/cas/we vs LPDDR2 CA encoding (cross-ref to §2.14)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 26 APB CSR Slave Protocol

**Status:** Skeleton — to be authored during MAS week

The APB CSR slave is the configuration and observation interface.

### 26.1 Protocol Compliance

---

APB3 (pslverr supported); APB4 (pstrb) optional.

### 26.2 Behavior

---

- Two-cycle access (SETUP + ENABLE) per the APB spec
- pslverr returns 1 for:
  - Writes to unimplemented schemes in ADDR\_MAP\_TUNING.scheme\_or
  - Writes to ranks beyond NUM\_RANKS (PASR registers, observation registers)
  - Writes to read-only registers
  - Reads from reserved addresses
- All registers are 32-bit; sub-32-bit accesses are aligned to 32-bit boundaries

### 26.3 CDC

---

The APB↔MC CDC lives in this FUB. See §1.3 and §2.18 csr\_apb\_fub for the CDC topology.

### 26.4 TODO

---

- pslverr decision tree

- APB-side timing diagram
- Sub-32-bit access policy
- CDC implementation detail

## 27 Register Map

**Status:** Skeleton — to be authored during MAS week

The full CSR map is in HAS §6.3. This MAS section is the **implementation-level** view: how each register is generated, what the reset value generation rule is, how runtime overrides are staged, and the canonical CSR YAML source.

### 27.1 Source of Truth

The CSR map is generated from `regs/ddr2_lpddr2_csrs.yaml` (PeakRDL- or RDL-style YAML). The build runs the generator to produce:

- `regs/ddr2_lpddr2_csr_pkg.sv` — SV package with offsets, masks, reset values
- `regs/ddr2_lpddr2_csr_block.sv` — the wrapped register file
- `regs/ddr2_lpddr2_csrs.h` — C header for software
- `regs/ddr2_lpddr2_csr_block_uvm.sv` — UVM register model

The MAS does not re-publish the bit-level register table; it documents the **generation pipeline** and the **runtime semantics** (quiet-point staging, etc.).

### 27.2 Staging vs. Live State

Every R/W field has two storage cells:

1. **APB-side staging register** — written immediately on the APB write
2. **MC-side live register** — copied from staging on the next quiet point

The staging→live commit is one MC cycle. Reads can return either side depending on the field — observation fields (OBS\_\*) read live state; configuration fields (\*\_TUNING) read staging by default but switch to live after CTRL.config\_apply is acknowledged.

## 27.3 TODO

---

- CSR YAML schema reference
- Generation script invocation
- Quiet-point handshake protocol detail
- Software access pattern (config-apply sequence)
- Generated UVM register model overview

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 28 Runtime Overrides and Quiet Points

**Status:** Skeleton — to be authored during MAS week

This section documents the runtime override mechanism — how a CSR write becomes a live datapath change without rebuilding or resetting.

### 28.1 Override Categories

---

Three flavors of override exist:

1. **Synthesized-MAX, runtime-active** — the silicon includes the maximum-capability logic; runtime CSR selects how much of it is active. Examples: SCHED\_TUNING.lookahead\_active, REFRESH\_TUNING.refresh\_defer\_active.
2. **Synthesized-set, runtime-mux** — the silicon includes a discrete set of variants; runtime CSR picks one. Examples: ADDR\_MAP\_TUNING.scheme\_or, REFRESH\_TUNING.refpb\_policy\_or.
3. **Pure runtime** — no silicon-area cost beyond the register itself. Examples: REFRESH\_TUNING.zqcs\_freq\_hz, INIT\_TUNING.zq\_retries.

## 28.2 Quiet-Point Sequence

---

1. Software writes the staging field via APB.
2. Software writes `CTRL.config_apply = 1`.
3. Controller drives `txn_queue` to drain, blocks new AXI accepts, completes any in-flight refresh.
4. `power_state_fub` asserts `quiet_point` for one cycle.
5. `csr_apb_fub` copies staging → live for every R/W field.
6. `STATUS.config_settled` rises.
7. Software reads `STATUS.config_settled`, clears `CTRL.config_apply`, and resumes traffic.

The drain phase is bounded — typically < 256 MC cycles for a healthy queue — and is the only operational disruption from a runtime override.

## 28.3 TODO

---

- Worst-case drain latency analysis
- Override propagation timing for each category
- Multi-write coalescing (if multiple staging writes are queued, commit them all at one quiet point)
- Software examples (init-time vs. runtime override patterns)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 29 Family-Wide Config-Bit Applicability

**Status:** Skeleton — to be authored during MAS week

The full family-wide config-bit registry is in HAS §5.4. This MAS section documents the implementation-level shape of those bits in **this controller specifically**.

### 29.1 Per-Bit Implementation Notes for DDR2 / LPDDR2

---

Bits flagged (reserved) in HAS §5.4 ECC\_TUNING are present in the CSR map at the documented offsets but read as zero and writes are ignored. The CSR slave does not raise `pslverr` on these —

software portability requires writing the family-wide config word to succeed on flavors that haven't implemented every bit.

Bits flagged DDR3+, DDR4+, LPDDR3+, LPDDR4-only likewise read as zero and ignore writes in this DDR2/LPDDR2 controller.

The 0xFF8 capability vector reports the build's actual capabilities; software queries this once at boot to decide which family-wide writes are meaningful.

## 29.2 Soft-N/A Behavior Examples

---

- Writing `SCHED_TUNING.bank_group_balance = 1` in DDR2 build: silently ignored, no `pslverr`. Read-back returns 0.
- Writing `POWER_TUNING.dpd_enable = 1` in DDR2 build: silently ignored. (DPD is LPDDR-only.)
- Reading `INIT_TUNING.cbt_enable` in any v1 controller: returns 0. (LPDDR4 only.)

## 29.3 TODO

---

- Per-bit implementation note for every family-wide bit
- Test plan for each soft-N/A bit
- Software bring-up sequence demonstrating capability-vector-gated writes

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

# 30 Initialization Sequence

**Status:** Skeleton — to be authored during MAS week

The init sequence from power-on through DRAM ready, as seen by software.

## 30.1 TODO

---

- Step-by-step software pseudocode
- Timing of each step (CSR write → controller state advance)
- Error-detection at each step
- Multi-rank init differences (per-rank ZQ, per-rank MR loads)

- Sim shortcut path (`SIM_INIT_SCALE > 1`)

## 31 Refresh and Power-State Programming

**Status:** Skeleton — to be authored during MAS week

Programming the refresh subsystem and the power-state FSM from software.

### 31.1 TODO

---

- Setting tREFI from CSR
- Selecting REFab vs REFpb per rank
- LPDDR2 PASR mask programming per rank
- Temperature-derate read-back (`MR4 / TEMP_DERATE_RANK`)
- ZQCS frequency tuning
- Entering / exiting self-refresh
- Entering / exiting Deep Power Down (LPDDR2)

## 32 Multi-Rank Programming

**Status:** Skeleton — to be authored during MAS week

The multi-rank-specific programming concerns: per-rank MR loads, per-rank PASR, cross-rank ODT, refresh fairness.

### 32.1 TODO

---

- Per-rank MR0/1/2/3 programming sequence
- Per-rank PASR programming



- ODT rule selection (ODT\_RULE\_MULTIRANK)
- Cross-rank ODT verification with the BFM
- Rank-disable semantics (RANK\_TUNING.rank\_enable\_mask)
- Multi-rank refresh schedule observation

## 33 Error Handling

**Status:** Skeleton — to be authored during MAS week

Error reporting paths and the software response model.

### 33.1 TODO

---

- Error categories: init failure, refresh-deadline miss, AXI queue overflow, DFI rddata-valid timeout
- IRQ topology (irq\_init\_done, irq\_init\_error, irq\_overflow)
- Clearing latched errors via CSR
- Diagnosis sequence: history register (STATUS\_HISTORY), debug observation FUBs
- Software recovery patterns (re-init, soft-reset, full reset)

## 34 Build-Time Configuration Reference

**Status:** Skeleton — to be authored during MAS week

The full build-time parameter table is in HAS §5.2. This MAS section is the **build-script** view: how each parameter is wired through filelists, includes, and synthesis scripts.

## 34.1 TODO

---

- Parameter override on synthesis command line
- Filelist composition per parameter combination
- rtl/includes/ddr2\_lpddr2\_config.svh macro definitions
- Conditional FUB inclusion (e.g., page\_predictor only when PAGE\_POLICY == HAPPY\_HYBRID)
- Per-memtype filelists

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

---

## 35 Runtime Configuration Reference

**Status:** Skeleton — to be authored during MAS week

The full runtime CSR is in HAS §6.3. This MAS section is the **driver-author** view: which fields can be tuned at runtime, in what order, and with what side-effects.

### 35.1 TODO

---

- Field-by-field runtime tuning cookbook
- Recommended bring-up sequence
- Recommended characterization sweep sequence
- Telemetry registers driving each tuning decision